

C++ 程序设计 完整复习

子冉一人 整理

2026 年 7 月 14 日

目录

1	绪论	8
1.1	C++ 发展历史	8
1.2	C 与 C++ 的区别	9
1.3	程序开发流程	10
1.4	编译环境	11
1.5	典型例题	11
1.6	巩固练习	12
2	C++ 入门	14
2.1	main 函数	14
2.2	输入输出 iostream	14
2.3	注释	15
2.4	控制结构初探	16
2.4.1	while 循环	16
2.4.2	for 循环	16
2.4.3	读取数量不定的输入数据	17
2.4.4	if 语句	17
2.5	类简介	18
2.6	典型例题	19
2.7	巩固练习	20
3	变量和基本类型	25
3.1	基本内置类型	25
3.1.1	算术类型	25
3.1.2	带符号与无符号	25

3.1.3	类型转换	26
3.1.4	字面值常量	26
3.2	变量	27
3.2.1	变量定义与初始化	27
3.2.2	变量作用域	27
3.3	复合类型	27
3.3.1	引用	28
3.3.2	指针	28
3.3.3	复合类型声明	29
3.4	const 限定符	29
3.4.1	const 基础	29
3.4.2	const 的引用	30
3.4.3	指针和 const	30
3.4.4	constexpr 和常量表达式	30
3.5	类型别名	31
3.6	auto 和 decltype	31
3.6.1	auto 类型说明符	31
3.6.2	decltype 类型指示符	31
3.7	典型例题	32
3.8	巩固练习	33
4	字符串、向量和数组	37
4.1	命名空间的 using 声明	37
4.2	string 类型	37
4.2.1	定义和初始化	37
4.2.2	string 上的操作	37
4.2.3	处理 string 中的字符	39
4.3	vector 类型	39
4.3.1	定义和初始化	39

4.3.2	vector 操作	40
4.4	迭代器	40
4.5	数组	41
4.5.1	定义和初始化	41
4.5.2	访问数组元素	42
4.5.3	指针和数组	42
4.6	多维数组	42
4.7	典型例题	43
4.8	巩固练习	44
5	运算符与表达式	48
5.1	基础概念	48
5.1.1	优先级和结合性	48
5.1.2	求值顺序	49
5.2	算术运算符	49
5.3	逻辑和关系运算符	50
5.4	赋值运算符	50
5.5	递增和递减运算符	51
5.6	成员访问运算符	51
5.7	条件运算符	51
5.8	位运算符	52
5.9	sizeof 运算符	52
5.10	逗号运算符	52
5.11	类型转换	53
5.11.1	隐式转换	53
5.11.2	显式转换（强制类型转换）	53
5.12	典型例题	54
5.13	巩固练习	55

6	语句	58
6.1	语句类型	58
6.2	条件语句	58
6.2.1	if 语句	58
6.2.2	switch 语句	59
6.3	循环语句	60
6.3.1	while 语句	60
6.3.2	for 语句	60
6.3.3	范围 for 语句 (C++11)	60
6.3.4	do-while 语句	61
6.4	跳转语句	61
6.5	异常处理	62
6.6	典型例题	63
6.7	巩固练习	64
7	函数	69
7.1	函数定义与调用	69
7.2	参数传递	70
7.2.1	值传递	70
7.2.2	引用传递	70
7.2.3	指针传递	71
7.2.4	const 形参	71
7.2.5	数组形参	71
7.2.6	可变参数	72
7.3	返回值	72
7.4	函数重载	73
7.5	默认参数	74
7.6	内联函数和 constexpr 函数	74
7.6.1	内联函数	74

7.6.2	constexpr 函数	75
7.7	调试帮助	75
7.7.1	assert 预处理宏	75
7.7.2	NDEBUG	75
7.8	函数指针	76
7.9	典型例题	77
7.10	巩固练习	78
8	IO 库	82
8.1	IO 类	82
8.2	标准输入输出 cin/cout	82
8.3	文件流 fstream	83
8.3.1	文件输出 ofstream	83
8.3.2	文件输入 ifstream	83
8.3.3	文件模式	84
8.4	字符串流 stringstream	85
8.5	格式化输入输出	86
8.6	IO 条件状态	87
8.7	典型例题	88
8.8	巩固练习	90
9	面向对象初探	94
9.1	类的基本概念	94
9.2	成员变量与成员函数	94
9.3	访问控制	95
9.4	构造函数	96
9.4.1	构造函数示例	96
9.4.2	类内初始值 (C++11)	97
9.4.3	拷贝构造函数	97
9.5	析构函数	98

9.6	this 指针	99
9.7	封装	100
9.8	继承初步	100
9.9	多态初步	101
9.10	典型例题	102
9.11	巩固练习	104
10	模拟试题	109
10.1	模拟试题一	109
10.2	模拟试题二	116

第 1 章 绪论

C++ 发展历史

C++ 是一门通用程序设计语言，由贝尔实验室的 Bjarne Stroustrup 于 1979 年开始设计，1985 年正式发布。C++ 以 C 语言为基础，最初被称为”C with Classes”（带类的 C），其面向对象思想主要受 Simula67 语言的启发。

C++ 标准演进

- 1979 年：C with Classes（C++ 前身）
- 1985 年：Cfront 编译器，The C++ Programming Language 第一版
- 1998 年：C++98，第一个 ISO 标准，引入 STL、异常、命名空间、RTTI、bool 类型
- 2011 年：C++11，重大更新，引入 auto、lambda、智能指针、右值引用、范围 for
- 2014 年：C++14，小幅完善
- 2017 年：C++17，进一步完善
- 2020 年：C++20，引入概念、协程、模块
- 2023 年：C++23

C 与 C++ 的区别

C 与 C++ 对比示例

C 语言（面向过程）：

```
#include <stdio.h>
#include <string.h>
struct Person {
    char name[20];
    int age;
};
struct Person p;    /* 全局变量 */
void init_person() { strcpy(p.name, "Yang"); p.age = 20; }
void show_person() { printf("%s: %d", p.name, p.age); }
int main() {
    init_person();
    show_person();
    return 0;
}
```

C++ 语言（面向对象）：

```
#include <iostream>
#include <string>
class Person {
    std::string name;    // 成员变量
    int age;
public:
    void init_person(); // 成员函数
    void show_person();
};
void Person::init_person() { name = "Yang"; age = 20; }
void Person::show_person() {
    std::cout << name << ": " << age;
```

```
}  
int main() {  
    Person p;          // 局部对象  
    p.init_person();    // 调用成员函数  
    p.show_person();  
    return 0;  
}
```

面向过程 vs 面向对象

特性	面向过程 (POP)	面向对象 (OOP)
构成	函数	对象
数据	与函数分离	与函数绑定 (在对象中)
安全	全局数据对所有函数可见	数据只能被所在对象的函数访问
方法	自顶向下	自底向上
重用	不易重用、不易扩展	易重用、易扩展
语言	C, Fortran, Pascal	C++, Java, Python, C#

程序开发流程

C/C++ 程序从源代码到可执行程序需经过四个阶段：

编译四步

1. **预处理**：处理以 # 开头的指令，如文件包含、宏定义、条件编译等，生成中间文件 (.i)
2. **编译**：将预处理后的源文件 (.c/.cpp) 转换为汇编文件 (.s/.asm)
3. **汇编**：将汇编文件转换为目标文件 (.o/.obj)
4. **链接**：将所有目标文件、预编译库文件、启动代码组合生成可执行程序 (.exe/.out)

编译环境

常用的 C++ 开发环境包括：

- **集成开发环境 (IDE)**：Visual Studio、Visual Studio Code、CLion、Dev-C++、Code::Blocks、Qt Creator
- **编译器**：GCC (g++)、Clang、MSVC (Microsoft Visual C++)、Intel C++ Compiler
- **在线编译器**：Replit、onlinegdb、Compiler Explorer、tutorialspoint

g++ 命令行编译

<code>g++ main.cpp</code>	# 编译生成 <code>a.exe</code> (Windows) / <code>a.out</code> (Linux)
<code>g++ main.cpp -o main</code>	# 指定输出文件名为 <code>main</code>
<code>g++ -E main.cpp > main.i</code>	# 只预处理，输出到 <code>main.i</code>
<code>g++ -S main.cpp</code>	# 预处理+编译，生成汇编 <code>main.s</code>
<code>g++ -c main.cpp</code>	# 预处理+编译+汇编，生成目标 <code>main.o</code>
<code>./main.exe</code>	# 执行程序

典型例题

例题 1.1

题目：简述 C++ 程序从源文件到可执行程序四个编译阶段。

参考答案与解析

答案：预处理 → 编译 → 汇编 → 链接

解析：

1. 预处理：处理 # 开头的预处理指令（如 #include、#define），展开宏和头文件
2. 编译：进行词法、语法、语义分析，将源代码转为汇编代码
3. 汇编：将汇编代码翻译成机器指令，生成目标文件（.o/.obj）
4. 链接：将目标文件与库文件等链接，生成最终可执行文件

例题 1.2

题目：说明面向过程编程与面向对象编程的主要区别。

参考答案与解析

答案：面向过程以函数为核心，数据与函数分离；面向对象以对象为核心，数据与操作封装在一起。

解析：面向过程采用自顶向下的设计，全局数据对所有函数可见，安全性较差，不易重用和扩展，代表语言有 C、Pascal。面向对象采用自底向上的设计，将数据和操作数据的函数封装为类，通过访问控制实现数据隐藏，支持继承和多态，易于重用和扩展，代表语言有 C++、Java、Python。

巩固练习

一、选择题

1. C++ 语言最初被称为_____。
A. C with Classes B. C with Objects C. C++0x D. Better C
2. 下列哪个不是 C++ 程序编译的四个阶段之一？
A. 预处理 B. 编译 C. 链接 D. 调试
3. C++ 语言的创始人是_____。
A. Dennis Ritchie B. Bjarne Stroustrup C. Brian Kernighan D. Ken Thompson
4. 下列命令中，只执行预处理步骤的是_____。
A. g++ -S main.cpp B. g++ -E main.cpp C. g++ -c main.cpp D. g++ main.cpp
5. 关于 C 与 C++ 的关系，下列说法正确的是_____。
A. C++ 是 C 的子集 B. C++ 基本是 C 的超集 C. C++ 与 C 完全无关 D. C++ 不能编译 C 程序
6. C++11 标准于哪一年发布？
A. 2003 B. 2011 C. 2014 D. 2017

参考答案与解析

答案： 1.A 2.D 3.B 4.B 5.B 6.B

解析：

1. C++ 最初称为”C with Classes”（带类的 C）
2. 编译四阶段为：预处理、编译、汇编、链接，调试不属于编译阶段
3. Bjarne Stroustrup 于 1979 年在贝尔实验室设计了 C++ 的前身
4. g++ -E 只执行预处理，-S 执行预处理 + 编译，-c 执行预处理 + 编译 + 汇编
5. C++ 基本是 C 的超集，可以运行约 99% 的 C 程序
6. C++11 于 2011 年发布，是 C++98 以来最大的更新

第2章 C++ 入门

main 函数

每个 C++ 程序必须包含一个名为 `main` 的函数，它是程序执行的唯一入口。

```
int main()
{
    return 0;
}
```

main 函数要点

- `main` 函数返回类型必须为 `int`
- 返回 0 表示程序正常执行，非 0 值通常表示错误类型
- C++ 以分号 (;) 表示语句结束
- 编译器会忽略换行符、空格符、制表符等空白符
- 函数定义包含：返回类型、函数名、(参数列表)、{函数体}

输入输出 `iostream`

C++ 通过标准库提供 I/O 功能，需包含 `<iostream>` 头文件。

```
#include <iostream>

int main()
{
    std::cout << "Hello world!" << std::endl;
    return 0;
}
```

IO 基础

- `std::cout`: 标准输出 (`ostream` 类型), 向控制台写入数据
- `std::cin`: 标准输入 (`istream` 类型), 从键盘读取数据
- `std::cerr`: 标准错误, 输出错误信息 (不缓冲)
- `std::clog`: 标准日志 (缓冲)
- `std::endl`: 操纵符, 换行并刷新缓冲区
- `<<`: 输出运算符, 将右侧值写到左侧输出对象
- `>>`: 输入运算符, 从左侧输入对象读入数据存入右侧对象
- `std::`: 命名空间前缀, `std` 是标准库的命名空间
- `::`: 作用域运算符

A+B Problem

```
#include <iostream>

int main()
{
    std::cout << "Enter two numbers:" << std::endl;
    int v1 = 0, v2 = 0;
    std::cin >> v1 >> v2;
    std::cout << "The sum of " << v1 << " and " << v2
                << " is " << v1 + v2 << std::endl;
    return 0;
}
```

注释

- 单行注释: `//` 到行末的内容
- 块注释 (C 风格): 以 `/*` 开始, 以 `*/` 结束, 可跨行, 但不能嵌套

```
// 这是单行注释
```

```
/*  
 * 这是多行注释  
 * 可以跨越多行  
 */
```

```
/* 注释不能 /* 嵌套 */ */ // 错误!
```

控制结构初探

while 循环

```
#include <iostream>  
int main()  
{  
    int sum = 0, val = 1;  
    while (val <= 10) {  
        sum += val;    // 等价于 sum = sum + val  
        ++val;        // 前缀递增: val = val + 1  
    }  
    std::cout << "Sum of 1 to 10 inclusive is " << sum << std::endl;  
    return 0;  
}
```

for 循环

```
#include <iostream>  
int main()  
{  
    int sum = 0;  
    for (int val = 1; val <= 10; ++val)  
        sum += val;  
    std::cout << "Sum of 1 to 10 inclusive is " << sum << std::endl;  
    return 0;  
}
```



```
}
```

for 循环说明

for 循环头的初始化语句中定义的变量仅在 for 循环内部存在，循环结束后无法访问。

```
for (int val = 1; val <= 10; ++val) // val 仅在循环内存在
    sum += val;
// std::cout << val; // 错误! val 在此处不存在
```

读取数量不定的输入数据

```
#include <iostream>
int main()
{
    int sum = 0, value = 0;
    while (std::cin >> value) // 读到 EOF 或无效输入时结束
        sum += value;
    std::cout << "Sum is: " << sum << std::endl;
    return 0;
}
```

流作为条件

istream 对象作为条件时检测流的状态：

- 遇到文件结束符 (EOF) 使流无效
- 遇到无效输入（如类型不匹配）使流无效
- Windows 下用 Ctrl+Z 输入 EOF，Linux/Mac 下用 Ctrl+D

if 语句

// 统计输入中每个值连续出现了几次

```
#include <iostream>
int main() {
    int currVal = 0, val = 0;
```

```
    if (std::cin >> currVal) {
        int cnt = 1;
        while (std::cin >> val) {
            if (val == currVal)
                ++cnt;
            else {
                std::cout << currVal << " occurs " << cnt
                    << " times" << std::endl;
                currVal = val;
                cnt = 1;
            }
        }
        std::cout << currVal << " occurs " << cnt
            << " times" << std::endl;
    }
    return 0;
}
```

类简介

类 (class) 用于自定义数据类型，可包含一组数据（成员变量）及可对该类型对象执行的操作（成员函数）。

使用类需要了解三件事

1. 类名是什么
2. 定义在哪个头文件中
3. 支持哪些操作

Sales_item 类的使用

```
#include <iostream>
#include "Sales_item.h"

int main()
```

```
{
    Sales_item item1, item2;
    std::cin >> item1 >> item2;
    // 先检测 ISBN 是否相同
    if (item1.isbn() == item2.isbn()) {
        std::cout << item1 + item2 << std::endl;
        return 0;
    } else {
        std::cerr << "Data must refer to same ISBN" << std::endl;
        return -1;
    }
}
```

说明：

- 标准库头文件用尖括号 <>，其他头文件用双引号 ""
- 成员运算符 .：左侧为对象，右侧为成员
- 函数调用运算符 ()：调用成员函数
- 运算符重载：对类对象重定义运算符行为

典型例题

例题 2.1

题目：编写程序，读取若干整数，输出它们的和。

参考答案与解析

```
#include <iostream>
int main()
{
    int sum = 0, value = 0;
    std::cout << "请输入若干整数 (Ctrl+Z 结束) : " << std::endl;
    while (std::cin >> value)
```

```
        sum += value;
    std::cout << "Sum is: " << sum << std::endl;
    return 0;
}
```

A. » B. « C. >< D. >«

3. `std::endl` 的作用是_____。

A. 只换行 B. 换行并刷新缓冲区 C. 只刷新缓冲区 D. 结束程序

4. 下列哪个是正确的单行注释？

A. # 注释 B. // 注释 C. - 注释 D. ' 注释

5. `std::cin >> v1 >> v2`; 等价于_____。

A. `std::cin >> v2 >> v1`; B. `(std::cin >> v1) >> v2`; C. `std::cin >> (v1 >> v2)`; D. `std::cin >> v1, v2`;

6. 关于 `main` 函数的返回值，下列说法正确的是_____。

A. 返回 1 表示正常 B. 返回 0 表示正常 C. 可以不返回值 D. 返回类型可以是 `void`

二、填空题

1. C++ 标准输入对象是 _____，标准输出对象是 _____。

2. `std` 是 C++ 标准库的_____。

3. 读取输入时遇到文件结束符的快捷键在 Windows 下是_____。

4. `for` 循环头中定义的变量其作用域是_____。

5. C 风格注释以 _____ 开始，以 _____ 结束，且不能_____。

三、简答题

1. 简述 `<<` 运算符在 `std::cout << "Hello" << std::endl`; 中的求值过程。

2. 说明 `while (std::cin >> value)` 中条件判断的工作原理。

四、综合应用题

1. 编写一个 C++ 程序，读取若干整数，统计并输出其中正数、负数和零的个数。

2. 编写程序，读取若干整数，输出每个连续重复值出现的次数。例如输入 42 42 42 7 7 9，输出"42 occurs 3 times"、"7 occurs 2 times"、"9 occurs 1 times"。

参考答案与解析

答案：选择题：1.B 2.B 3.B 4.B 5.B 6.B

解析：

1. `main` 是程序执行的唯一入口
2. `<<` 是输出运算符，`>>` 是输入运算符
3. `endl` 换行并刷新缓冲区，`\n` 只换行不刷新
4. `//` 是单行注释，`#` 是预处理指令符号
5. `>>` 左结合，等价于 `(std::cin >> v1) >> v2;`
6. `main` 返回 `int`，返回 0 表示正常

填空题答案：

1. `std::cin`; `std::cout`
2. 命名空间 (namespace)
3. `Ctrl+Z`
4. 仅在该 `for` 循环内部
5. `/*`; `*/`; 嵌套

简答题答案：

1. `<<` 运算符左结合，先执行 `std::cout << "Hello"`，返回 `std::cout`，再执行 `std::cout << std::endl`。
2. `>>` 运算符返回其左侧的 `istream` 对象，`istream` 对象在条件中检测流的状态。遇到 EOF 或无效输入时流变为无效，条件为假，循环结束。

编程题参考代码:

第 1 题:

```
#include <iostream>
int main() {
    int num, pos = 0, neg = 0, zero = 0;
    while (std::cin >> num) {
        if (num > 0) ++pos;
        else if (num < 0) ++neg;
        else ++zero;
    }
    std::cout << "正数: " << pos << std::endl;
    std::cout << "负数: " << neg << std::endl;
    std::cout << "零: " << zero << std::endl;
    return 0;
}
```

第 2 题:

```
#include <iostream>
int main() {
    int currVal = 0, val = 0;
    if (std::cin >> currVal) {
        int cnt = 1;
        while (std::cin >> val) {
            if (val == currVal) ++cnt;
            else {
                std::cout << currVal << " occurs "
                    << cnt << " times" << std::endl;
                currVal = val;
                cnt = 1;
            }
        }
        std::cout << currVal << " occurs "
            << cnt << " times" << std::endl;
    }
```

```
    }  
    return 0;  
}
```


第3章 变量和基本类型

基本内置类型

C++ 定义了丰富的算术类型，分为整型（包括字符型和布尔型）和浮点型。

算术类型

基本算术类型

类型	含义	最小位	字节	有效数字
bool	布尔型	8 位	1 字节	—
char	字符型	8 位	1 字节	—
short	短整型	16 位	2 字节	5 位
int	整型	16 位	4 字节	10 位
long	长整型	32 位	4/8 字节	19 位
long long	长长整型	64 位	8 字节	19 位
float	单精度浮点	32 位	4 字节	6~7 位
double	双精度浮点	64 位	8 字节	15~16 位
long double	扩展精度	64 位	16 字节	18~19 位

C/C++ 标准保证：1 = char ≤ short ≤ int ≤ long ≤ long long

带符号与无符号

整型分带符号 (signed) 和无符号 (unsigned) 两种：

- 带符号类型可表示正数、负数和 0
- 无符号类型仅能表示大于等于 0 的值
- unsigned int 可缩写为 unsigned
- int、short、long 默认都是带符号的

- char 默认是否带符号视编译器而定

类型转换

隐式类型转换规则

- char/short $\rightarrow$ int (整型提升)
- 窄 $\rightarrow$ 宽转换: int $\rightarrow$ unsigned $\rightarrow$ long $\rightarrow$ long long
- 浮点转换: float $\rightarrow$ double $\rightarrow$ long double
- 非 bool $\rightarrow$ bool: 0 转 false, 非 0 转 true
- bool $\rightarrow$ 非 bool: false 转 0, true 转 1
- 浮点转整型会舍弃小数部分 (舍位)

无符号类型溢出陷阱

```
unsigned u = 10;
int i = -42;
std::cout << i + i << std::endl; // 输出 -84
std::cout << u + i << std::endl; // 若 int 占32位, 输出 4294967264
```

当无符号类型与带符号类型运算时, 带符号数会自动转换为无符号数。-42 转为无符号后变为一个很大的正数。

字面值常量

- 整型字面值: 20 (十进制)、024 (八进制)、0x14 (十六进制)
- 浮点字面值: 3.14、3.14e0、0.、.001
- 字符字面值: 'a'、'\n' (换行)
- 字符串字面值: "Hello" (末尾自动加 '\0')
- 布尔字面值: true、false
- 指针字面值: nullptr

变量

变量定义与初始化

初始化方式

- 默认初始化: `int a;` (内置类型在函数外默认初始化为 0, 函数内值未定义)
- 拷贝初始化: `int a = 0;` (使用 `=`)
- 直接初始化: `int a(0);` (使用括号)
- 列表初始化 (C++11): `int a{0};` (使用花括号)

列表初始化

```
int units_sold = 0;
int units_sold(0);
int units_sold{0};          // C++11 列表初始化
int units_sold = {0};       // 列表初始化

long double ld = 3.1415926536;
int a{ld};                  // 错误: 列表初始化不允许窄化转换
int b = ld;                  // 正确: 会窄化, 但编译器可能警告
```

变量作用域

作用域规则

- 全局作用域: 定义在所有花括号之外, 整个程序可见
- 块作用域: 定义在花括号内, 仅在该块内可见
- 内层屏蔽外层: 内层同名变量会屏蔽外层变量
- 使用作用域运算符 `::`: 可访问全局变量: `::value`

复合类型

引用

引用（reference）是为对象起的别名。

```
int ival = 1024;
int &refVal = ival; // refVal 是 ival 的别名
refVal = 2;         // 等价于 ival = 2
int &refVal2;       // 错误！引用必须初始化
```

引用要点

- 引用必须初始化，且初始值必须是一个对象（非字面值）
- 引用一经绑定就不能改变绑定对象
- 引用类型必须与绑定对象类型严格一致
- 引用不是对象，不能定义引用的引用

指针

指针（pointer）是存储对象地址的对象。

```
int ival = 42;
int *p = &ival; // p 存放 ival 的地址
cout << *p;     // 解引用，输出 42
*p = 0;         // 通过指针修改 ival 的值为 0
```

指针要点

- 指针本身就是一个对象，可以赋值和拷贝
- 指针可以不初始化（但建议初始化为 `nullptr`）
- `&`：取地址运算符
- `*`：解引用运算符
- `nullptr`：空指针字面值（C++11）
- 指针类型必须与所指对象类型严格一致

- 不能定义指向引用的指针（引用不是对象）
- 空指针不能解引用

指针与引用对比

```
int i = 42;
int &r = i;    // 引用: r 是 i 的别名
int *p = &i;   // 指针: p 存储 i 的地址
*p = 50;      // 通过指针修改 i 为 50
r = 60;       // 通过引用修改 i 为 60

int j = 100;
p = &j;       // 指针可以改变指向, 现在 p 指向 j
// r = j;     // 这是赋值! 将 j 的值赋给 i, 而非改变 r 的绑定
```

复合类型声明

```
int i = 1024, *p = &i, &r = i;
// i 是 int, p 是 int 指针, r 是 int 引用
```

```
int *p1, p2;    // p1 是 int 指针, p2 是 int (不是指针!)
int *p1, *p2;   // p1 和 p2 都是 int 指针
```

const 限定符

const 定义一种值不能被改变的变量。

const 基础

```
const int bufSize = 512; // 编译期常量
bufSize = 1024;         // 错误! 不能修改 const 对象
```

const 要点

- const 对象必须初始化
- const 对象不能被修改

- 默认情况下，`const` 对象仅在当前文件有效
- 多个文件共享 `const` 对象需加 `extern` 声明

const 的引用

```
const int ci = 1024;
const int &r1 = ci;    // 正确：引用及其对象都是 const
int &r2 = ci;         // 错误！不能用非 const 引用绑定 const 对象

int i = 42;
const int &r3 = i;     // 正确：const 引用可绑定非 const 对象
const int &r4 = 42;    // 正确：const 引用可绑定字面值
const int &r5 = r3 * 2; // 正确：const 引用可绑定表达式
```

const 引用

`const` 引用可以绑定到非 `const` 对象、字面值、表达式。但非 `const` 引用不能绑定到 `const` 对象。

指针和 const

```
const int ci = 1024;
const int *p1 = &ci; // 指向 const int 的指针：不能通过 p1 修改 ci
int *const p2 = &i;  // const 指针：p2 本身是常量，不能改变指向
const int *const p3 = &ci; // 都不能改变
```

顶层 const 与底层 const

- 顶层 `const` (top-level): 指针本身是常量 (`int *const p`)
- 底层 `const` (low-level): 指针所指对象是常量 (`const int *p`)
- 顶层 `const` 在拷贝时可被忽略，底层 `const` 必须类型匹配

constexpr 和常量表达式

```
const int sz = 10;           // sz 是常量表达式
constexpr int sz2 = sz + 1; // C++11: 编译期验证是否为常量表达式
```

类型别名

```
typedef double wages;      // wages 是 double 的别名
typedef wages base, *p;    // base 是 double, p 是 double*

using SI = Sales_item;     // C++11 别名声明
```

auto 和 decltype

auto 类型说明符

auto 让编译器自动推断变量类型（C++11）。

```
auto i = 42;      // i 是 int
auto d = 3.14;    // d 是 double
auto s = "hello"; // s 是 const char*
auto &r = i;       // r 是 int&（引用）
auto *p = &i;     // p 是 int*（指针）
```

auto 推断规则

- auto 会忽略顶层 const，保留底层 const
- 要保留顶层 const 需显式写出：const auto ci = i;
- 引用和指针要显式写出：auto &r = i;

decltype 类型指示符

decltype 从表达式推断类型（C++11）。

```
int a = 3, b = 4;
decltype(a) c;      // c 是 int（a 的类型）
decltype(a + b) d;  // d 是 int（a+b 的类型）
decltype((a)) e;    // e 是 int&（双层括号表示引用！）
```

decltype 规则

- `decltype(var)`: 返回 `var` 的类型（包括顶层 `const` 和引用）
- `decltype((var))`: 双层括号永远返回引用类型
- `decltype(expr)`: 根据表达式结果推断

典型例题

例题 3.1

题目：判断下列代码哪些是合法的。

```
int i = 42;  
(a) const int &r = i;  
(b) int &const r2 = i;  
(c) const int *p = &i;  
(d) int *const p2 = &i;
```

参考答案与解析

答案：(a) 合法，(b) 不合法，(c) 合法，(d) 合法

解析：

- (a) `const` 引用可绑定非 `const` 对象，合法
- (b) 引用本身不是对象，不能加 `const` 限定，不合法
- (c) 指向 `const int` 的指针，不能通过 `p` 修改 `i`，但 `p` 可改变指向，合法
- (d) `const` 指针，`p2` 不能改变指向，但可通过 `p2` 修改 `i`，合法

例题 3.2

题目：下列程序的输出结果是什么？

```
#include <iostream>  
int main() {  
    int i = 0, &r = i;
```



```
auto a = r;
const int ci = i, &cr = ci;
auto b = ci;
auto c = cr;
a = 42; b = 42; c = 42;
std::cout << i << " " << ci << std::endl;
return 0;
}
```

参考答案与解析

答案：0 0

解析：a、b、c 都是 auto 推断的普通 int 变量（auto 忽略顶层 const 和引用），它们的修改不影响原始变量 i 和 ci，故输出 0 0。

巩固练习

一、选择题

1. 下列类型中，能表示负数的是_____。

A. unsigned int B. unsigned char C. int D. unsigned long

2. 下列代码的输出是_____。

```
unsigned u = 10, u2 = 42;
std::cout << u2 - u << std::endl;
```

A. 32 B. 4294967264 C. -32 D. 编译错误

3. 下列哪种初始化方式可以防止窄化转换？

A. int a = 3.14; B. int a(3.14); C. int a{3.14}; D. int a = (int)3.14;

4. 下列关于引用的说法，错误的是_____。

A. 引用必须初始化 B. 引用一经绑定不能改变 C. 引用是对象的别名 D. 引用本身是一个对象

5. 下列代码中，p 的类型是_____。

```
int i = 42;
const int *p = &i;
```

- A. const 指针 B. 指向 const int 的指针 C. int 指针 D. const int

6. decltype((i)) 的结果是 (i 是 int 变量) _____。

- A. int B. int& C. const int D. int*

二、填空题

1. 在 32 位系统中，sizeof(int) 通常是_____ 字节。
2. int i = 3.14; 中 i 的值是_____。
3. 定义指向 int 的指针 p 并初始化为空指针的语句是_____。
4. 顶层 const 表示_____ 是常量，底层 const 表示_____ 是常量。
5. const int *p 和 int *const p 中，_____ 是 const 指针。

三、简答题

1. 简述 const int *p、int *const p、const int *const p 三者的区别。
2. 说明 auto 和 decltype 在类型推断时的主要区别。

四、综合应用题

1. 编写程序，使用 auto 和范围 for 循环，统计一个字符串中元音字母的个数。
2. 解释下列代码中每条语句的作用，并指出哪些是合法的：

```
int i = 0;
int &r = i;
const int ci = i;
```

```
const int &cr = ci;  
r = ci;  
ci = r;  
cr = i;
```

参考答案与解析

答案：选择题：1.C 2.A 3.C 4.D 5.B 6.B

解析：

1. 只有带符号类型能表示负数，int 是带符号的
2. $u2-u=42-10=32$ ，都是 unsigned，结果仍为 unsigned，值为 32
3. 列表初始化 {} 不允许窄化转换
4. 引用不是对象，是对象的别名
5. `const int *p` 是指向 `const int` 的指针
6. `decltype((i))` 双层括号返回引用类型

填空题答案：

1. 4
2. 3
3. `int *p = nullptr;`
4. 指针本身；指针所指对象
5. `int *const p`

简答题答案：

1. `const int *p` 是指向常量的指针，不能通过 `p` 修改所指对象，但 `p` 可以改变指向。`int *const p` 是常量指针，`p` 本身是常量不能改变指向，但可通过 `p` 修改所指对象。`const int *const p` 两者都不能改变。

2. `auto` 会忽略顶层 `const` 和引用, `decltype` 会保留顶层 `const` 和引用。 `decltype((var))` 双层括号返回引用。

编程题参考代码:

第 1 题:

```
#include <iostream>
#include <string>
using std::string;
int main() {
    string s;
    std::getline(std::cin, s);
    int vowel_cnt = 0;
    for (auto c : s) {
        if (c == 'a' || c == 'e' || c == 'i' ||
            c == 'o' || c == 'u' ||
            c == 'A' || c == 'E' || c == 'I' ||
            c == 'O' || c == 'U')
            ++vowel_cnt;
    }
    std::cout << "元音字母个数: " << vowel_cnt << std::endl;
    return 0;
}
```

第 2 题: `r = ci`; 合法, 将 `ci` 的值赋给 `r` 引用的 `i`。 `ci = r`; 不合法, `ci` 是常量不能修改。 `cr = i`; 不合法, `cr` 是 `const` 引用不能修改所绑定的对象。

第4章 字符串、向量和数组

命名空间的 using 声明

```
using std::cin;      // using 声明，后续可直接使用 cin
using std::cout;
using std::endl;
cin >> i;           // 等价于 std::cin >> i
using namespace std; // 引入 std 命名空间所有名字（不推荐在头文件中使用）
```

string 类型

string 表示可变长的字符序列，需包含 <string> 头文件。

定义和初始化

```
string s1;           // 默认初始化为空字符串
string s2(s1);        // s2 是 s1 的副本
string s2 = s1;       // 等价于 s2(s1)
string s3("value");   // s3 是 "value" 的副本
string s4 = "value";  // 等价于 s4("value")
string s5(5, 'c');    // s5 = "ccccc", 5个字符 c
```

string 上的操作

string 常用操作

操作	含义
<code>os << s</code>	将 s 写到输出流 os
<code>is >> s</code>	从 is 读取字符串（以空白分隔）
<code>getline(is, s)</code>	从 is 读取一行存入 s
<code>s.empty()</code>	s 为空返回 true
<code>s.size()</code>	返回 s 中字符个数
<code>s[n]</code>	返回 s 中第 n 个字符的引用
<code>s1 + s2</code>	返回连接后的结果
<code>s1 = s2</code>	用 s2 的副本替代 s1
<code>s1 == s2</code>	判断相等
<code><, <=, >, >=</code>	字典序比较

string 读写

```
string s;
cin >> s;           // 跳过前导空白，遇到空白停止
cout << s << endl;

string s1, s2;
cin >> s1 >> s2;    // 依次读取两个"单词"
cout << s1 << s2 << endl;

// 读取一整行（可包含空格）
string line;
while (getline(cin, line))
    if (!line.empty())
        cout << line << endl;
```

string::size_type

string 的 size() 函数返回 string::size_type 类型，这是一个无符号整型。避免将其与 int 混用，特别是当 int 为负值时，与无符号数比较会出问题。

```
string line;  
auto len = line.size(); // 类型为 string::size_type
```

处理 string 中的字符

// 范围 for 遍历每个字符

```
string str("C++");  
for (auto c : str)  
    cout << c << endl;
```

// 使用引用修改字符

```
string s("Hello World!!!");  
for (auto &c : s)  
    c = toupper(c); // c 是引用，修改 s 中的字符  
cout << s << endl; // 输出 HELLO WORLD!!!
```

vector 类型

vector 表示可变长度的同类型对象的集合，是标准模板库 (STL) 的容器，需包含 `<vector>` 头文件。

定义和初始化

```
vector<int> ivec;           // 空 vector  
vector<int> ivec2(ivec);    // 拷贝 ivec 的元素  
vector<int> ivec3 = ivec;    // 等价于 ivec3(ivec)  
vector<int> ivec4 = {0, 1, 2}; // 列表初始化 (C++11)  
vector<string> svec{"C", "C++"};  
vector<int> ivec5(10, -1);   // 10个-1  
vector<int> ivec6(10);       // 10个0
```

vector 初始化注意

- `vector<int> v1(10);` 是 10 个元素，每个值为 0
- `vector<int> v2{10};` 是 1 个元素，值为 10（列表初始化）

- `vector<int> v3(10, 1);` 是 10 个 1
- `vector<int> v4{10, 1};` 是 2 个元素：10 和 1

vector 操作

```
vector<int> v;  
v.push_back(3);      // 尾部添加元素 3  
v.empty();           // 是否为空  
v.size();            // 元素个数  
v[0];                // 下标访问（不检查越界）  
v1 = v2;             // 拷贝赋值  
v1 == v2;            // 相等比较  
for (auto &i : v)     // 范围 for  
    i *= 2;
```

vector 下标越界

- 下标运算符 `[]` 不检查越界
- 越界访问是未定义行为，可能导致缓冲区溢出
- 下标类型应为 `vector<int>::size_type`
- 不能用下标添加元素，只能用 `push_back`

迭代器

迭代器类似指针，用于访问容器中的元素。

```
string s("hello");  
for (auto it = s.begin(); it != s.end() && !isspace(*it); ++it)  
    *it = toupper(*it);  
// s 变为 "Hello"
```

```
vector<int> v{1, 2, 3, 4, 5};  
auto it = v.begin();    // 指向第一个元素
```



```
*it = 10;           // 修改第一个元素为 10
++it;               // 前进到下一个元素
```

迭代器操作

操作	含义
*iter	返回迭代器所指元素的引用
iter->mem	解引用并访问成员 mem
++iter	前进到下一个元素
--iter	后退到上一个元素
iter1 == iter2	判断两个迭代器是否相等
iter1 != iter2	判断是否不等
v.begin()	首元素迭代器
v.end()	尾后迭代器（不指向任何元素）

迭代器分类

- 正向迭代器：只能前进
- 双向迭代器：可前进可后退（list、set、map）
- 随机访问迭代器：支持任意位置访问（vector、array、deque）

数组

数组是固定长度的同类型元素集合，编译期确定长度。

定义和初始化

```
int arr[10];           // 10 个 int，值未定义
int arr2[3] = {1, 2, 3};
int arr3[5] = {1, 2};  // 剩余元素初始化为 0
int arr4[] = {1, 2, 3}; // 长度由初值列表决定，为 3
char arr5[6] = "Daniel"; // 错误！没有空间放 '\0'
char arr6[7] = "Daniel"; // 正确
```

访问数组元素

```
int arr[5] = {10, 20, 30, 40, 50};
for (size_t i = 0; i < 5; ++i)
    cout << arr[i] << " ";
// 输出 10 20 30 40 50
```

指针和数组

```
int arr[] = {1, 2, 3, 4, 5};
int *p = arr;          // p 指向 arr[0]
int *p2 = &arr[0];     // 等价于 p
int i = *p;            // i = 1
++p;                   // p 指向 arr[1]
int j = *p;            // j = 2

int *end = arr + 5;    // 尾后指针
for (int *q = arr; q != end; ++q)
    cout << *q << " ";
```

数组与指针

- 数组名在很多表达式中会自动转换为指向首元素的指针
- `int arr[5]; auto a = arr;` 中 `a` 是 `int*`（不是数组！）
- `decltype(arr)` 返回数组类型（不退化）
- `begin(arr)` 和 `end(arr)` 返回首指针和尾后指针（C++11）

多维数组

```
int arr[3][4];          // 3行4列的二维数组
int arr2[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};
```

```
int arr3[2][3] = {1, 2, 3, 4, 5, 6}; // 等价写法

// 范围 for 遍历
for (auto &row : arr2)    // row 是引用，避免数组退化为指针
    for (auto col : row)
        cout << col << " ";
```

多维数组遍历注意

使用范围 for 遍历多维数组时，除了最内层循环外，所有循环变量都必须是引用类型，否则数组会退化为指针，无法遍历。

典型例题

例题 4.1

题目：下列初始化哪些是正确的？

- (a) `vector<int> v1(10);`
- (b) `vector<int> v2{10};`
- (c) `vector<int> v3(10, 1);`
- (d) `vector<int> v4{10, 1};`

参考答案与解析

答案：全部正确，但含义不同。

解析：

- (a) v1 有 10 个元素，每个值为 0
- (b) v2 有 1 个元素，值为 10
- (c) v3 有 10 个元素，每个值为 1
- (d) v4 有 2 个元素，值为 10 和 1

例题 4.2

题目：编写程序，从标准输入读取若干整数，存入 vector，然后输出每对相邻整数的和。

参考答案与解析

```
#include <iostream>
#include <vector>
using std::vector;
using std::cin; using std::cout; using std::endl;

int main() {
    vector<int> v;
    int num;
    while (cin >> num)
        v.push_back(num);
    for (decltype(v.size()) i = 0; i + 1 < v.size(); i += 2)
        cout << v[i] + v[i+1] << " ";
    if (v.size() % 2 != 0)
        cout << v[v.size()-1];
    cout << endl;
    return 0;
}
```

巩固练习

一、选择题

- 下列哪个操作可以读取包含空格的一整行？
A. `cin >> s` B. `getline(cin, s)` C. `cin.getline(s)` D. `cin.get(s)`
- `vector<int> v(10);` 中 `v` 有多少个元素？
A. 0 B. 1 C. 10 D. 编译错误
- `vector<int> v{10};` 中 `v` 有多少个元素？

- A. 0 B. 1 C. 10 D. 编译错误

4. 下列代码中，p 指向哪个元素？

```
int arr[] = {1, 2, 3, 4, 5};  
int *p = arr + 2;
```

- A. arr[0] B. arr[1] C. arr[2] D. arr[3]

5. `string s("hello"); s.size()` 的返回值类型是_____。

- A. int B. unsigned C. `string::size_type` D. `size_t`

6. 下列关于 vector 的说法，错误的是_____。

- A. 可以动态增长 B. 下标可以添加元素 C. 下标访问不检查越界 D. 支持范围 for 循环

二、填空题

1. `string s = "hello";` 则 `s.size()` 的值是_____。
2. 使用范围 for 修改 string 中的字符，循环变量应声明为_____ 类型。
3. `vector<int> v{1, 2, 3};` 中 `v[1]` 的值是_____。
4. 迭代器 `v.end()` 指向的位置是_____。
5. `int arr[5];` 中 `sizeof(arr)` 的值是_____ 字节（假设 int 占 4 字节）。

三、简答题

1. 说明 `cin >> s` 和 `getline(cin, s)` 的区别。
2. 为什么使用范围 for 遍历多维数组时，外层循环变量必须是引用？

四、综合应用题

1. 编写程序，读取一组整数，存入 vector，然后输出其中最大的元素及其出现次数。

2. 编写程序，将一个字符串中的所有小写字母转换为大写字母，其他字符保持不变。

参考答案与解析

答案：选择题：1.B 2.C 3.B 4.C 5.C 6.B

解析：

1. `getline` 读取一整行，>> 以空白分隔
2. `v(10)` 是 10 个元素
3. `v{10}` 是列表初始化，1 个元素
4. `arr + 2` 指向 `arr[2]`
5. `size()` 返回 `string::size_type`
6. 不能用下标添加元素，只能用 `push_back`

填空题答案：

1. 5
2. 引用 (`auto &c`)
3. 2
4. 尾后位置（不指向任何元素）
5. 20

编程题参考代码：

第 1 题：

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> v;
    int num;
```

```
while (cin >> num)
    v.push_back(num);
if (v.empty()) return 0;
int max_val = v[0];
int cnt = 1;
for (size_t i = 1; i < v.size(); ++i) {
    if (v[i] > max_val) {
        max_val = v[i];
        cnt = 1;
    } else if (v[i] == max_val) {
        ++cnt;
    }
}
cout << "最大值: " << max_val
    << ", 出现次数: " << cnt << endl;
return 0;
}
```

第 2 题:

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s;
    getline(cin, s);
    for (auto &c : s)
        if (c >= 'a' && c <= 'z')
            c = c - 'a' + 'A';
    cout << s << endl;
    return 0;
}
```

第 5 章 运算符与表达式

基础概念

表达式由运算符和运算对象组成。C++ 有 9 类共 54 个运算符。

运算符分类

- 算术运算符：+ - * / %
- 关系运算符：== != < <= > >=
- 逻辑运算符：! && ||
- 自增自减：++ --
- 位运算符：~& | ^ << >>
- 赋值与复合赋值：= += -= *= /= %=
- 成员访问：[] * & . ->
- 其他：sizeof , (type) ?: f()

优先级和结合性

优先级与结合性

- 优先级高的运算符先计算
- 优先级相同的运算符按结合性决定顺序
- 左结合（从左向右）：除赋值外的大多数二元运算符
- 右结合（从右向左）：赋值运算符、一元运算符、条件运算符
- 括号可改变优先级

求值顺序

求值顺序

只有四种运算符规定了求值顺序：&&、||、?:、，
其他运算符的操作数求值顺序未定义，因此不要在同一表达式中既修改又使用同一变量：

```
cout << i << ++i;    // 未定义行为
j = i * i++;          // 未定义行为
```

算术运算符

算术运算符优先级

运算符	功能	用法	优先级
+ -	一元正负	+a -a	3
* / %	乘、除、取余	a*b a/b a%b	5
+ -	加、减	a+b a-b	6

整数除法和取余

- 两个整数相除结果仍为整数（舍位，舍弃小数部分）
- C++11 规定整数除法向 0 取整
- 取余运算符 % 操作数必须为整数
- $m\%n$ 的符号与 m 相同
- 0 不能用作除法或取余的右操作数

```
5 / 2 = 2      5 % 2 = 1
-5 / 2 = -2    -5 % 2 = -1
5 / -2 = -2    5 % -2 = 1
-5 / -2 = -2   -5 % -2 = -1
```

逻辑和关系运算符

关系运算符

`==` `!=` `<` `<=` `>` `>=`, 结果为 `bool` 类型。[易错] `==` 不要写成 `=!`

逻辑运算符

- `!a`: 逻辑非（一元）
- `a && b`: 逻辑与，短路求值（`a` 为假时不再求 `b`）
- `a || b`: 逻辑或，短路求值（`a` 为真时不再求 `b`）

短路求值

```
// 安全访问：先检查 index 合法性
if (index != s.size() && !isspace(s[index]))
    // 只有 index 合法时才会访问 s[index]

// 常用模式：先检查指针非空
if (p && p->value > 0)
    // 只有 p 非空时才会访问 p->value
```

赋值运算符

赋值运算符

- 赋值运算符 `=` 是右结合
- 赋值返回左侧运算对象
- 复合赋值：`+=` `-=` `*=` `/=` `%=` `<<=` `>>=` `&=` `|=` `^=`
- `a += b` 等价于 `a = a + b`

```
int i;
i = 0;           // 赋值
i = i + 1;       // 等价于 i += 1 或 ++i
```

```
a = b = c = 0; // 右结合，等价于 a = (b = (c = 0))
```

递增和递减运算符

前缀与后缀

- ++a（前缀）：先加 1，返回新值
- a++（后缀）：返回旧值，再加 1
- 前缀版本更高效（无需保存旧值）
- 优先级：后缀高于前缀

```
int i = 0, j;  
j = ++i; // j = 1, i = 1（先加再赋值）  
j = i++; // j = 1, i = 2（先赋值再加）
```

*p++ 的含义

```
*p++; // 等价于 *(p++): 先解引用，p 再后缀自增  
// 后缀++ 优先级高于*，所以先执行 p++  
// p++ 返回 p 的旧值（指向当前元素），再解引用  
// 然后 p 前进到下一个元素
```

成员访问运算符

```
string s = "hello", *p = &s;  
s.size(); // 通过对象访问成员  
p->size(); // 通过指针访问成员，等价于 (*p).size()
```

条件运算符

```
cond ? expr1 : expr2;
```

```
int max_val = (a > b) ? a : b; // 取较大值
```

位运算符

位运算符

运算符	名称	功能
~a	按位取反	0 变 1, 1 变 0
a & b	按位与	两位都为 1 才为 1
a b	按位或	有一位为 1 就为 1
a ^ b	按位异或	两位不同为 1
a << n	左移	左移 n 位, 低位补 0
a >> n	右移	无符号补 0, 有符号补符号位

位运算示例

```
unsigned char a = 0b1010;  // 10
unsigned char b = 0b1100;  // 12

a & b  // = 0b1000 = 8
a | b  // = 0b1110 = 14
a ^ b  // = 0b0110 = 6
~a     // = 0b11110101 = 245 (8位)
a << 2 // = 0b101000 = 40
```

sizeof 运算符

sizeof 返回类型或对象所占字节数。

```
sizeof(int)      // 4
sizeof(double)   // 8
sizeof(arr)      // 整个数组的大小
sizeof(arr) / sizeof(arr[0]) // 数组元素个数
```

逗号运算符

a, b 先求 a, 再求 b, 结果为 b。

```
for (int i = 0, j = 10; i < j; ++i, --j)
    // ...
```

类型转换

隐式转换

常见隐式转换

- 整型提升: `char`、`short` $\rightarrow$ `int`
- 算术转换: 窄 $\rightarrow$ 宽
- 数组 $\rightarrow$ 指针: 数组名退化为指针
- 指针转换: `0` $\rightarrow$ 空指针, 非 `const` $\rightarrow$ `const`
- `bool` 转换: `0` $\rightarrow$ `false`, 非 `0` $\rightarrow$ `true`

显式转换（强制类型转换）

C++ 四种强制转换

- `static_cast<type>(expr)`: 通用类型转换
- `const_cast<type>(expr)`: 去掉 `const` 性质
- `reinterpret_cast<type>(expr)`: 重新解释位模式
- `dynamic_cast<type>(expr)`: 安全的多态向下转换

```
double d = 3.14;
int i = static_cast<int>(d); // i = 3

const char *pc = "hello";
char *p = const_cast<char *>(pc); // 去掉 const

// C 风格转换（不推荐）
int j = (int)d;
```

```
int k = int(d);
```

典型例题

例题 5.1

题目：计算下列表达式的值。

```
int a = 5, b = 3;
```

- (a) a / b
- (b) $a \% b$
- (c) $-a / b$
- (d) $a \ll 2$

参考答案与解析

答案： (a) 1 (b) 2 (c) -1 (d) 20

解析：

- (a) 整数除法 $5/3=1$ （舍位）
- (b) $5\%3=2$
- (c) C++11 整数除法向 0 取整， $-5/3=-1$
- (d) 5 的二进制 101，左移 2 位为 10100=20

例题 5.2

题目：下列代码的输出是什么？

```
int i = 1, j = 2;  
int k = i++ + ++j;  
cout << i << " " << j << " " << k << endl;
```

参考答案与解析

答案： 2 3 4

解析： $i++$ 先返回旧值 1，然后 i 变为 2。 $++j$ 先加 1， j 变为 3，返回新值 3。 $k = 1 + 3 = 4$ 。

巩固练习

一、选择题

1. 下列运算符中，优先级最高的是_____。
A. + B. * C. ++ D. ==
2. 下列表达式的值是_____。
7 / 2
A. 3.5 B. 3 C. 4 D. 编译错误
3. `int a = 5; a % 2` 的值是_____。
A. 2 B. 1 C. 2.5 D. 编译错误
4. 下列哪个运算符有短路求值特性？
A. & B. && C. | D. +
5. `int i = 3; int j = i++;` 则 `i` 和 `j` 的值分别是_____。
A. 3, 3 B. 4, 3 C. 3, 4 D. 4, 4
6. 下列代码的输出是_____。

```
int x = 10;  
cout << (x >> 2) << endl;
```


A. 2 B. 3 C. 4 D. 5

二、填空题

1. `5 / 2` 的结果是_____，`5.0 / 2` 的结果是_____。
2. `int a = 10; a += 5;` 执行后 `a` 的值是_____。
3. 逻辑与运算符 `&&` 的短路特性是：当左侧为_____ 时，不再计算右侧。
4. `10 & 6` 的结果是_____（二进制 `1010 & 0110`）。

5. `sizeof(int)` 在 32 位系统中通常是_____ 字节。

三、简答题

1. 简述前缀自增 `++i` 和后缀自增 `i++` 的区别。
2. 说明 `&` 作为取地址运算符和按位与运算符的区别。

四、综合应用题

1. 编写程序，输入一个整数，判断其是否为 2 的幂次方（使用位运算）。
2. 编写程序，输入一个整数 `n`，使用位运算计算 `n` 的二进制表示中 1 的个数。

参考答案与解析

答案：选择题：1.C 2.B 3.B 4.B 5.B 6.A

解析：

1. 后缀自增 `++` 优先级高于前缀和二元运算符
2. 整数除法舍位， $7/2=3$
3. $5\%2=1$
4. `&&` 逻辑与有短路特性，`&` 按位与没有
5. `i++` 返回旧值 3 给 `j`，然后 `i` 变 4
6. 10 的二进制 1010，右移 2 位为 $10=2$

填空题答案：

1. 2; 2.5
2. 15
3. false（假）
4. 2

5. 4

编程题参考代码:

第 1 题:

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    // 2的幂次方的二进制只有一个1
    // n > 0 且 (n & (n-1)) == 0
    if (n > 0 && (n & (n - 1)) == 0)
        cout << n << " 是2的幂次方" << endl;
    else
        cout << n << " 不是2的幂次方" << endl;
    return 0;
}
```

第 2 题:

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    int count = 0;
    while (n) {
        count += (n & 1);
        n >>= 1;
    }
    cout << "1的个数: " << count << endl;
    return 0;
}
```

第6章 语句

语句类型

C++ 语句分类

- 表达式语句：表达式后加分号
- 空语句：只含一个分号
- 复合语句（块）：花括号括起的一组语句
- 条件语句：if、switch
- 循环语句：while、for、do-while
- 跳转语句：break、continue、goto、return
- 异常处理：try/catch/throw

条件语句

if 语句

```
if (条件)
    语句1
else
    语句2
```

```
// 嵌套 if-else (多路分支)
if (score >= 90) grade = 'A';
else if (score >= 80) grade = 'B';
else if (score >= 70) grade = 'C';
else if (score >= 60) grade = 'D';
```

```
else grade = 'F';
```

悬垂 else

else 与同层级中最近的、未匹配的 if 配对。建议用花括号明确配对关系。

switch 语句

```
switch (整型表达式) {  
    case 常量1:  
        语句1  
        break;    // 不加 break 会"贯穿"  
    case 常量2:  
        语句2  
        break;  
    default:  
        默认语句  
}
```

switch 要点

- switch 表达式必须是整型、枚举或相容类型
- case 值必须是整型常量表达式，且互不相同
- 不加 break 会继续执行下一个 case（贯穿）
- default 分支可选
- case 标签内定义变量需用花括号括起

switch 示例

```
// 统计元音字母个数  
unsigned aCnt = 0, eCnt = 0, iCnt = 0, oCnt = 0, uCnt = 0;  
char ch;  
while (cin >> ch) {  
    switch (ch) {  
        case 'a': case 'A': ++aCnt; break;
```

```
        case 'e': case 'E': ++eCnt; break;
        case 'i': case 'I': ++iCnt; break;
        case 'o': case 'O': ++oCnt; break;
        case 'u': case 'U': ++uCnt; break;
    }
}
```

循环语句

while 语句

```
while (条件)
    语句
// 先测试后执行，可能一次都不执行
```

for 语句

```
for (初始化; 条件; 更新)
    语句
```

```
// 等价于：
{
    初始化;
    while (条件) {
        语句
        更新;
    }
}
```

范围 for 语句 (C++11)

```
for (声明 : 序列)
    语句
```

```
string s("hello");
```

```
for (auto c : s)           // 只读遍历
    cout << c;
for (auto &c : s)          // 引用，可修改
    c = toupper(c);
```

do-while 语句

```
do
    语句
while (条件);
// 先执行后测试，至少执行一次
```

do-while 示例

```
// 反复读取直到输入合法值
int n;
do {
    cout << "请输入一个正整数: ";
    cin >> n;
} while (n <= 0);
```

跳转语句

跳转语句

- **break:** 跳出当前 switch 或循环
- **continue:** 结束当前迭代，开始下一次循环
- **goto:** 跳转到标签语句（不推荐使用）
- **return:** 结束函数并返回值

```
// break 示例
for (int i = 0; i < 10; ++i) {
    if (i == 5) break;           // i=5 时跳出循环
    cout << i << " ";          // 输出 0 1 2 3 4
```

```
}
```

```
// continue 示例
```

```
for (int i = 0; i < 10; ++i) {  
    if (i % 2 == 0) continue; // 跳过偶数  
    cout << i << " ";      // 输出 1 3 5 7 9  
}
```

异常处理

```
try {  
    // 可能抛出异常的代码  
    if (error_condition)  
        throw runtime_error("错误信息");  
} catch (runtime_error err) {  
    // 处理异常  
    cout << err.what() << endl;  
}
```

异常处理要点

- throw: 抛出异常
- try: 包裹可能抛出异常的代码
- catch: 捕获并处理异常
- 标准异常类定义在 `<stdexcept>` 头文件中
- 常用异常: `runtime_error`、`logic_error`、`overflow_error`
- `err.what()` 返回异常信息字符串

异常处理示例

```
#include <iostream>  
#include <stdexcept>  
using namespace std;
```

```
int divide(int a, int b) {
    if (b == 0)
        throw runtime_error("除数不能为0");
    return a / b;
}

int main() {
    int a, b;
    cin >> a >> b;
    try {
        cout << divide(a, b) << endl;
    } catch (runtime_error &e) {
        cout << "错误: " << e.what() << endl;
    }
    return 0;
}
```

典型例题

例题 6.1

题目：下列代码的输出是什么？

```
for (int i = 1; i <= 5; ++i) {
    if (i == 3) continue;
    if (i == 5) break;
    cout << i << " ";
}
```

参考答案与解析

答案：1 2 4

解析：i=1 输出 1；i=2 输出 2；i=3 遇 continue 跳过；i=4 输出 4；i=5 遇 break 跳出。

例题 6.2

题目：编写程序，使用 switch 语句将百分制成绩转换为五级制。

参考答案与解析

```
#include <iostream>
using namespace std;
int main() {
    int score;
    cin >> score;
    char grade;
    switch (score / 10) {
        case 10:
        case 9: grade = 'A'; break;
        case 8: grade = 'B'; break;
        case 7: grade = 'C'; break;
        case 6: grade = 'D'; break;
        default: grade = 'F';
    }
    cout << "等级: " << grade << endl;
    return 0;
}
```

巩固练习

一、选择题

1. 下列关于 switch 语句的说法，正确的是_____。

A. case 后可以用浮点数 B. case 值可以重复 C. 不加 break 会贯穿 D. default 必须存在

2. 下列循环中，至少执行一次的是_____。

A. while B. for C. do-while D. 范围 for

3. break 语句的作用是_____。

- A. 结束当前迭代 B. 跳出循环或 switch C. 结束函数 D. 跳转到标签

4. 下列代码输出_____。

```
int i = 0;
while (i < 3) {
    cout << i << " ";
    i++;
}
```

- A. 0 1 2 B. 1 2 3 C. 0 1 2 3 D. 1 2

5. 下列代码输出_____。

```
for (int i = 0; i < 5; ++i) {
    if (i % 2) continue;
    cout << i;
}
```

- A. 01234 B. 024 C. 135 D. 1234

6. 下列关于异常处理的说法，错误的是_____。

- A. throw 抛出异常 B. try 包裹可能出错的代码 C. catch 捕获异常 D. 异常必须立即捕获

二、填空题

- else 总是与同层级中_____ 的、未匹配的 if 配对。
- do-while 循环至少执行_____ 次循环体。
- continue 语句的作用是结束_____，开始下一次循环。
- switch 语句中，case 后必须是_____ 常量表达式。
- C++ 标准异常类定义在_____ 头文件中。

三、简答题

1. 简述 break 和 continue 的区别。
2. 说明 switch 语句中 break 的作用，不加 break 会发生什么？

四、综合应用题

1. 编写程序，输入一个正整数 n，判断它是否为素数。
2. 编写一个简单的计算器程序，输入两个数和运算符（+、-、*、/），使用 switch 语句实现。除数为 0 时使用异常处理。

参考答案与解析

答案：选择题：1.C 2.C 3.B 4.A 5.B 6.D

解析：

1. switch 中 case 值必须是整型常量，不加 break 会贯穿
2. do-while 先执行后测试，至少执行一次
3. break 跳出循环或 switch
4. while 循环从 i=0 开始，输出 0 1 2
5. i%2 为真（奇数）时 continue 跳过，输出偶数 024
6. 异常可以不立即捕获，会向上传播

填空题答案：

1. 最近
2. 一
3. 当前迭代
4. 整型
5. <stdexcept>

编程题参考代码:

第 1 题:

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cin >> n;
    if (n < 2) { cout << "不是素数" << endl; return 0; }
    bool is_prime = true;
    for (int i = 2; i * i <= n; ++i) {
        if (n % i == 0) {
            is_prime = false;
            break;
        }
    }
    cout << (is_prime ? "是素数" : "不是素数") << endl;
    return 0;
}
```

第 2 题:

```
#include <iostream>
#include <stdexcept>
using namespace std;
int main() {
    double a, b;
    char op;
    cin >> a >> b >> op;
    try {
        double result;
        switch (op) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
```

```
        case '/':
            if (b == 0) throw runtime_error("除数不能为0");
            result = a / b; break;
        default: throw runtime_error("未知运算符");
    }
    cout << a << " " << op << " " << b
         << " = " << result << endl;
} catch (runtime_error &e) {
    cout << "错误: " << e.what() << endl;
}
return 0;
}
```

第7章 函数

函数定义与调用

函数定义

```
返回类型 函数名(形参列表) {  
    函数体  
    return 返回值;  
}
```

- **函数声明**（函数原型）：给出返回类型、函数名、形参类型
- **函数定义**：包含函数体
- 函数必须先声明后使用，定义本身也是声明
- 形参名在声明中可省略

```
// 声明  
int max_int(int i, int j);    // 带形参名  
int max_int(int, int);       // 省略形参名  
  
// 定义  
int max_int(int i, int j) {  
    return i > j ? i : j;  
}
```

形参与实参

- **形参 (parameter)**：函数定义中的参数，调用时其值来自实参
- **实参 (argument)**：函数调用时传递的值
- 实参按位置初始化形参

- 实参的求值顺序未定义
- 实参数量和类型必须与形参匹配

参数传递

值传递

```
void reset(int val) {  
    val = 0;    // 只修改了局部副本，不影响实参  
}  
  
int main() {  
    int n = 42;  
    reset(n);    // n 仍然是 42  
}
```

值传递

- 形参是实参的副本
- 修改形参不影响实参
- 适合小对象（int、double 等）

引用传递

```
void reset(int &val) {  
    val = 0;    // 直接修改实参  
}  
  
int main() {  
    int n = 42;  
    reset(n);    // n 变为 0  
}
```

引用传递

- 形参是实参的别名，不拷贝
- 修改形参即修改实参
- 适合大对象，避免拷贝开销
- 不修改实参时用 `const` 引用：`void f(const string &s)`

指针传递

```
void reset(int *p) {  
    *p = 0;    // 修改 p 所指对象  
}  
  
int main() {  
    int n = 42;  
    reset(&n); // 传地址, n 变为 0  
}
```

const 形参

const 形参与实参匹配

- 使用 `const` 引用可接受 `const` 对象、非 `const` 对象、字面值
- 非 `const` 引用只能接受同类型的非 `const` 左值
- 尽量使用 `const` 引用

数组形参

```
// 三种等价写法，都传递首元素指针  
void print(const int*);  
void print(const int[]);  
void print(const int[10]); // 维度 10 只是提示，不检查  
  
// 使用标记（C 风格字符串）  
void print(const char *p) {
```

```
    while (*p) cout << *p++;  
}  
  
// 使用标准库 begin/end  
void print(const int *beg, const int *end) {  
    while (beg != end) cout << *beg++ << " ";  
}  
  
// 传递大小参数  
void print(const int ia[], size_t size) {  
    for (size_t i = 0; i < size; ++i)  
        cout << ia[i] << " ";  
}
```

可变参数

```
// initializer_list (同类型参数)  
void error_msg(initializer_list<string> il) {  
    for (auto beg = il.begin(); beg != il.end(); ++beg)  
        cout << *beg << " ";  
}  
error_msg({"hello", "world"});  
  
// 省略符 (C 风格, 不安全)  
int printf(const char *format, ...);
```

返回值

返回类型

- 返回类型可以是 void (无返回值)
- return 语句结束函数执行
- void 函数可使用 return; 提前退出
- 不要返回局部变量的引用或指针

- 可以返回局部变量的副本（值）
- 引用返回：返回左值（如赋值运算符）

返回引用

```
// 返回引用，可作为左值
char &get_val(string &str, string::size_type ix) {
    return str[ix];
}

int main() {
    string s("hello");
    get_val(s, 0) = 'H'; // 将 s[0] 改为 'H'
    cout << s;           // 输出 Hello
}
```

函数重载

函数重载：同名函数有不同的形参列表（参数数量或类型不同）。

```
void print(const char *cp);           // 打印 C 风格字符串
void print(const int *beg, const int *end); // 打印整数区间
void print(const int ia[], size_t size); // 打印数组

print("hello");                       // 调用第一个
int arr[] = {1, 2, 3};
print(begin(arr), end(arr));           // 调用第二个
print(arr, 3);                         // 调用第三个
```

重载要点

- 重载函数的形参列表必须不同（数量或类型）
- 返回类型不参与重载判断
- const 形参与非 const 形参可以重载
- 顶层 const 不参与重载，底层 const 参与

- `int f(int)` 和 `int f(const int)` 是同一函数
- `int f(int*)` 和 `int f(const int*)` 是不同函数

默认参数

// 默认参数必须从右向左连续提供

```
string screen(int ht = 24, int wid = 80, char background = ' ');
```

```
screen();                // screen(24, 80, ' ')
screen(66);              // screen(66, 80, ' ')
screen(66, 256);         // screen(66, 256, ' ')
screen(66, 256, '#');    // screen(66, 256, '#')
```

默认参数规则

- 默认参数必须从右向左连续提供
- 有默认参数的形参右侧所有形参都必须有默认值
- 函数声明中给出默认值后，定义中不能重复指定
- 局部变量不能作为默认参数

内联函数和 `constexpr` 函数

内联函数

```
inline int max(int a, int b) {
    return a > b ? a : b;
}
```

内联函数

- `inline` 建议编译器在调用处展开（不一定采纳）
- 适合规模小、流程直接、频繁调用的函数
- 内联函数通常定义在头文件中

- 避免在循环、递归中使用

constexpr 函数

```
constexpr int factorial(int n) {  
    return n <= 1 ? 1 : n * factorial(n - 1);  
}  
  
constexpr int f5 = factorial(5); // 编译期计算, f5 = 120
```

constexpr 函数

- 用于常量表达式
- 返回类型和形参类型必须是字面值类型
- 函数体必须有且仅有一条 return 语句
- 可以返回非常量（在非常量上下文中调用时）

调试帮助

assert 预处理宏

```
#include <cassert>  
assert(expr); // expr 为假时终止程序
```

NDEBUG

```
// 定义 NDEBUG 后 assert 不起作用  
// 编译时: g++ -D NDEBUG main.cpp
```

```
// 自定义调试输出
```

```
#ifndef NDEBUG  
    cerr << "debug: " << __FILE__ << ":" << __LINE__  
          << " in " << __func__ << endl;  
#endif
```

调试宏

- `__FILE__`: 当前文件名
- `__LINE__`: 当前行号
- `__func__`: 当前函数名
- `__TIME__`: 编译时间
- `__DATE__`: 编译日期

函数指针

// 函数指针

```
int (*pf)(int, int); // pf 指向返回 int、参数为 (int,int) 的函数
```

```
int add(int a, int b) { return a + b; }
```

```
pf = add; // pf 指向 add
```

```
int result = pf(3, 4); // 等价于 add(3, 4)
```

// 用作函数参数

```
void use(int (*f)(int, int), int a, int b) {
```

```
    cout << f(a, b);
```

```
}
```

// 等价语法

```
void use(int f(int, int), int a, int b);
```

// 类型别名简化

```
typedef int (*FuncPtr)(int, int);
```

```
using FuncPtr = int (*)(int, int);
```

典型例题

例题 7.1

题目：以下代码的输出是什么？

```
void swap(int a, int b) {  
    int temp = a; a = b; b = temp;  
}  
void swap2(int *a, int *b) {  
    int temp = *a; *a = *b; *b = temp;  
}  
void swap3(int &a, int &b) {  
    int temp = a; a = b; b = temp;  
}  
int main() {  
    int x = 1, y = 2;  
    swap(x, y);    cout << x << y;    // 输出 12  
    swap2(&x, &y); cout << x << y;    // 输出 21  
    swap3(x, y);   cout << x << y;    // 输出 12  
}
```

参考答案与解析

答案：122112

解析：

- swap 是值传递，不改变实参，输出 12
- swap2 是指针传递，交换后 x=2,y=1，输出 21
- swap3 是引用传递，再次交换后 x=1,y=2，输出 12

例题 7.2

题目：下列哪些重载是合法的？

(a) int f(int x); int f(int y);

```
(b) int f(int x);    double f(int x);  
(c) int f(int x);    int f(const int x);  
(d) int f(int x);    int f(int x, int y);
```

参考答案与解析

答案：(a) 不合法, (b) 不合法, (c) 不合法, (d) 合法

解析：

- (a) 形参名不同不影响重载，是同一函数
- (b) 返回类型不参与重载判断
- (c) 顶层 const 不参与重载，是同一函数
- (d) 参数数量不同，合法重载

巩固练习

一、选择题

1. 下列哪种参数传递方式可以修改实参？

- A. 值传递 B. const 引用传递 C. 指针传递 D. const 指针传递

2. 下列关于函数重载的说法，正确的是_____。

- A. 返回类型不同即可重载 B. 形参列表不同才能重载 C. 形参名不同即可重载 D. 函数体不同即可重载

3. 默认参数必须从_____开始连续提供。

- A. 左 B. 右 C. 中间 D. 任意位置

4. 下列代码的输出是_____。

```
int f(int n) { return n * n; }  
int f(int &n) { return ++n; }
```

- A. 可以重载 B. 不能重载 C. 取决于调用 D. 编译错误

5. `inline` 关键字的作用是_____。

- A. 强制内联 B. 建议内联 C. 禁止内联 D. 使函数成为常量

6. 下列代码的输出是_____。

```
void g(int &x) { x = 10; }
int main() {
    int a = 5;
    g(a);
    cout << a;
}
```

- A. 5 B. 10 C. 编译错误 D. 未定义行为

二、填空题

1. 函数声明也称为函数_____。
2. 值传递时，形参是实参的_____。
3. 引用传递时，形参是实参的_____。
4. 顶层 `const`_____ 参与重载，底层 `const`_____ 参与重载。（填“参与”或“不参与”）
5. `assert` 宏定义在_____ 头文件中。

三、简答题

1. 比较值传递、引用传递和指针传递的优缺点。
2. 说明内联函数和普通函数的区别。

四、综合应用题

1. 编写一个函数 `swap`，使用引用传递交换两个整数的值。编写主函数测试。
2. 编写重载函数 `max`，分别求两个 `int`、两个 `double`、三个 `int` 的最大值。

参考答案与解析

答案：选择题：1.C 2.B 3.B 4.B 5.B 6.B

解析：

1. 值传递和 `const` 引用传递不能修改实参，指针和普通引用可以
2. 重载要求形参列表不同，返回类型不参与判断
3. 默认参数从右向左连续提供
4. 顶层 `const` (`const int`) 不参与重载，这两个函数冲突
5. `inline` 只是建议编译器内联展开
6. 引用传递，`a` 被修改为 10

填空题答案：

1. 原型
2. 副本（拷贝）
3. 别名
4. 不参与；参与
5. `<cassert>`

编程题参考代码：

第 1 题：

```
#include <iostream>
using namespace std;
void swap(int &a, int &b) {
    int temp = a; a = b; b = temp;
}
int main() {
    int x, y;
    cin >> x >> y;
```



```
    cout << "交换前: " << x << " " << y << endl;
    swap(x, y);
    cout << "交换后: " << x << " " << y << endl;
    return 0;
}
```

第 2 题:

```
#include <iostream>
using namespace std;
int max(int a, int b) { return a > b ? a : b; }
double max(double a, double b) { return a > b ? a : b; }
int max(int a, int b, int c) {
    return max(max(a, b), c);
}
int main() {
    cout << max(3, 5) << endl;
    cout << max(3.14, 2.71) << endl;
    cout << max(1, 5, 3) << endl;
    return 0;
}
```

第8章 IO 库

IO 类

标准库 IO 类层次

头文件	类型	用途
<iostream>	istream, ostream	标准 I/O cin, cout, cerr, clog
<fstream>	ifstream, ofstream	文件 I/O
<sstream>	istringstream, ostringstream	字符串流 I/O

- ifstream 继承自 istream
- istringstream 继承自 istream
- ofstream 继承自 ostream
- ostringstream 继承自 ostream
- fstream 继承自 istream
- stringstream 继承自 istream

标准输入输出 cin/cout

```
#include <iostream>
using namespace std;
```

```
int main() {
    int a;
    double d;
    string s;
```

```
    cin >> a >> d >> s;    // 输入跳过前导空白
    cout << a << " " << d << " " << s << endl;
    cerr << "错误信息" << endl;    // 不缓冲
    clog << "日志信息" << endl;    // 缓冲
    return 0;
}
```

文件流 fstream

文件输出 ofstream

```
#include <fstream>
using namespace std;

int main() {
    ofstream fout("data.txt");    // 打开文件，不存在则创建
    if (fout) {
        fout << "Hello" << endl;
        fout << 42 << " " << 3.14 << endl;
        fout.close();    // 显式关闭（出作用域也会自动关闭）
    }
    return 0;
}
```

文件输入 ifstream

```
#include <fstream>
#include <iostream>
#include <string>
using namespace std;

int main() {
    ifstream fin("data.txt");
    if (fin.is_open()) {
        string line;
```

```
        while (getline(fin, line))    // 逐行读取
            cout << line << endl;
        fin.close();
    } else {
        cerr << "无法打开文件" << endl;
    }
    return 0;
}
```

文件模式

文件模式常量

模式	含义
<code>ios::in</code>	以读方式打开
<code>ios::out</code>	以写方式打开
<code>ios::app</code>	追加模式（在末尾写）
<code>ios::ate</code>	打开后定位到末尾
<code>ios::trunc</code>	截断文件（清空）
<code>ios::binary</code>	二进制模式

- `ofstream` 默认模式为 `out | trunc`（清空写）
- `ifstream` 默认模式为 `in`
- `fstream` 默认模式为 `out | in`
- 追加写：`ofstream f("file", ios::app);`
- 模式可用按位或 `|` 组合

文件模式示例

```
// 清空写（ofstream 默认行为）
ofstream out1("file");                // out | trunc

// 追加写
```

```
ofstream out2("file", ios::app);           // out | app
ofstream out3("file", ios::out | ios::app); // 显式指定

// 读+写（不清空）
fstream file("file", ios::in | ios::out);

// 二进制写
ofstream bin("file.bin", ios::out | ios::binary);
```

字符串流 sstream

```
#include <sstream>
#include <iostream>
using namespace std;

// ostreamstream: 将多个值拼成字符串
ostreamstream oss;
oss << "Name: " << "Zhang" << ", Age: " << 20;
string result = oss.str(); // result = "Name: Zhang, Age: 20"

// istringstream: 从字符串中提取数据
string line = "2025 Zhang 95.5";
istringstream iss(line);
int year;
string name;
double score;
iss >> year >> name >> score; // 分别提取

// 常用：字符串与数值转换
int n = 42;
ostreamstream convert;
convert << n;
string s = convert.str(); // "42"
```

```
istringstream back("3.14");  
double d;  
back >> d;  // d = 3.14
```

字符串流常用场景

- 字符串与其他类型转换
- 逐行读取后分词
- 格式化输出
- `.str()` 获取流内容字符串
- `.str(s)` 将字符串 `s` 设为流内容

格式化输入输出

常用格式控制

- `endl`: 换行并刷新
- `setw(n)`: 设置字段宽度（下一个输出项）
- `setprecision(n)`: 设置精度
- `setfill(c)`: 设置填充字符
- `fixed`: 固定小数点表示
- `scientific`: 科学计数法
- `hex`、`oct`、`dec`: 进制
- `setbase(n)`: 设置进制（0/8/10/16）

格式化输出

```
#include <iostream>  
#include <iomanip>
```

```
using namespace std;

int main() {
    double d = 3.14159265;

    cout << fixed << setprecision(2) << d << endl;
    // 输出 3.14

    cout << setw(10) << setfill('*') << 42 << endl;
    // 输出 *****42

    cout << hex << 255 << " " << oct << 8 << endl;
    // 输出 ff 10

    cout << left << setw(10) << "Name" << 100 << endl;
    // 输出 Name      100

    return 0;
}
```

IO 条件状态

IO 状态

状态	含义
goodbit (0)	无错误
badbit	流已破坏（不可恢复）
failbit	操作失败（可恢复）
eofbit	到达文件末尾

- `s.good()`: 流是否有效
- `s.eof()`: 是否到达 EOF
- `s.fail()`: 是否失败

- `s.bad()`: 是否损坏
- `s.clear()`: 清除所有错误状态
- `s.clear(state)`: 设置指定状态
- `s.setstate(state)`: 设置状态

IO 状态恢复

```
int val;
while (cin >> val) {
    // 处理 val
}
// 循环结束时 cin 处于失败状态
if (cin.eof()) cout << "到达文件末尾" << endl;
if (cin.fail()) {
    cin.clear(); // 清除错误状态
    string bad_input;
    cin >> bad_input; // 读取并丢弃错误输入
    cout << "输入错误: " << bad_input << endl;
}
```

典型例题

例题 8.1

题目：编写程序，将一个文本文件中的所有行读取出来并输出到另一个文件，每行前加行号。

参考答案与解析

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
int main() {
```



```
    ifstream fin("input.txt");
    ofstream fout("output.txt");
    if (!fin || !fout) {
        cerr << "文件打开失败" << endl;
        return 1;
    }
    string line;
    int line_no = 1;
    while (getline(fin, line))
        fout << line_no++ << ": " << line << endl;
    fin.close();
    fout.close();
    return 0;
}
```

例题 8.2

题目：使用 stringstream 实现字符串与整数的转换。

参考答案与解析

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std;

int str_to_int(const string &s) {
    istringstream iss(s);
    int n;
    iss >> n;
    return n;
}

string int_to_str(int n) {
    ostringstream oss;
```

```
    oss << n;
    return oss.str();
}

int main() {
    cout << str_to_int("42") + 1 << endl;    // 43
    cout << int_to_str(100) + "x" << endl;    // 100x
    return 0;
}
```

巩固练习

一、选择题

1. 文件流 `ifstream` 默认的打开模式是_____。
A. `ios::out` B. `ios::in` C. `ios::app` D. `ios::trunc`
2. `ofstream` 打开文件时，默认行为是_____。
A. 追加写 B. 清空写 C. 只读 D. 报错
3. 下列哪个头文件包含 `stringstream`?
A. `<iostream>` B. `<fstream>` C. `<sstream>` D. `<string>`
4. 流状态 `eofbit` 表示_____。
A. 流损坏 B. 到达文件末尾 C. 操作失败 D. 流有效
5. 下列代码的输出是_____。

```
cout << setw(5) << setfill('0') << 42;
```

- A. 42 B. 00042 C. 42000 D. 42
6. 清除流错误状态使用_____。
A. `clear()` B. `close()` C. `flush()` D. `reset()`

二、填空题

1. `getline(cin, s)` 从标准输入读取_____ 存入 `s`。
2. `cerr` 是_____ 缓冲的, `clog` 是_____ 缓冲的。
3. 追加写文件应使用模式_____。
4. `setprecision(n)` 设置浮点数的_____。
5. 字符串流 `ostringstream` 的 `str()` 方法返回_____。

三、简答题

1. 说明 `cin >> s` 和 `getline(cin, s)` 的区别。
2. 简述 IO 流的四种条件状态及其含义。

四、综合应用题

1. 编写程序, 从文件 `numbers.txt` 中读取若干整数, 计算它们的平均值, 并将结果输出到 `result.txt`。
2. 编写程序, 使用 `stringstream` 将一行用逗号分隔的数字 (如 "1,2,3,4,5") 提取并求和。

参考答案与解析

答案: 选择题: 1.B 2.B 3.C 4.B 5.B 6.A

解析:

1. `ifstream` 默认以读方式打开
2. `ofstream` 默认清空写 (`out | trunc`)
3. `istringstream` 定义在 `<sstream>` 中
4. `eofbit` 表示到达文件末尾
5. `setw(5)` 宽度 5, `setfill('0')` 填充 0, 输出 00042

6. clear() 清除所有错误状态

填空题答案:

1. 一整行
2. 不; 有
3. ios::app
4. 精度 (有效数字位数)
5. 流中的字符串内容

编程题参考代码:

第 1 题:

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
    ifstream fin("numbers.txt");
    if (!fin) { cerr << "打开失败" << endl; return 1; }
    int n, sum = 0, count = 0;
    while (fin >> n) { sum += n; ++count; }
    fin.close();
    ofstream fout("result.txt");
    if (count > 0)
        fout << "平均值: " << (double)sum / count << endl;
    else
        fout << "无数据" << endl;
    fout.close();
    return 0;
}
```

第 2 题:

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std;
int main() {
    string line;
    getline(cin, line);
    istringstream iss(line);
    string token;
    int sum = 0;
    while (getline(iss, token, ',')) {
        istringstream num_stream(token);
        int n;
        num_stream >> n;
        sum += n;
    }
    cout << "总和: " << sum << endl;
    return 0;
}
```

第9章 面向对象初探

类的基本概念

类 (class) 是一种自定义数据类型，将数据（成员变量）和操作数据的函数（成员函数）封装在一起。

```
class Contact {  
private:  
    string name;          // 成员变量（私有）  
    long long phone;  
public:  
    void setName(string); // 成员函数（公有）  
    void setPhone(long long);  
    void show();  
}; // 注意类定义末尾的分号！  
  
void Contact::setName(string str) { name = str; }  
void Contact::setPhone(long long no) { phone = no; }  
void Contact::show() {  
    cout << name << ": " << phone << endl;  
}
```

成员变量与成员函数

类的成员

- 成员变量：类中定义的数据
- 成员函数：类中定义的函数，可访问类中的所有成员
- :: 作用域运算符：在类外定义成员函数时指明所属类
- . 成员运算符：通过对象访问公有成员

- -> 箭头运算符：通过对象指针访问公有成员

类的使用

```
int main() {  
    Contact contact;           // 定义对象  
    contact.setName("YangXiaodong");  
    contact.setPhone(12345678901LL);  
    contact.show();            // YangXiaodong: 12345678901  
  
    Contact *pc = new Contact; // 动态创建对象  
    pc->setName("Jobs");  
    pc->show();  
    delete pc;  
}
```

访问控制

访问控制修饰符

访问修饰符	类内	类外
public	可访问	可访问
private	可访问	不可访问
protected	可访问	不可访问（派生类可访问）

- class 默认访问权限是 private
- struct 默认访问权限是 public
- 成员变量常声明为 private（数据隐藏）
- 成员函数常声明为 public（公有接口）

class 与 struct 的区别

```
struct S { int a; };           // a 默认 public
class C { int a; };           // a 默认 private

S s = {10};                   // 合法, a 是 public
// C c = {10};                // 错误, a 是 private
```

构造函数

构造函数是创建对象时自动调用的成员函数，用于初始化对象。

构造函数要点

- 函数名与类名相同
- 没有返回类型（连 void 都不能写）
- 可以重载
- 无参数的构造函数称为默认构造函数
- 未定义任何构造函数时，编译器自动提供默认构造函数
- 一旦定义了任何构造函数，编译器不再自动提供默认构造函数
- C++11: `Contact() = default;` 显式要求编译器生成默认构造函数

构造函数示例

```
class Contact {
private:
    string name;
    long long phone;
public:
    Contact();                // 默认构造函数
    Contact(const string &str, long long no);    // 带参构造函数
    Contact(const string &str = "", long long no = 0); // 带默认参数
```



```
};
```

```
// 构造函数初始值列表（推荐，更高效）
```

```
Contact::Contact() : phone(0) { }
```

```
Contact::Contact(const string &str, long long no)
```

```
    : name(str), phone(no)    // 初始值列表
```

```
{
```

```
}
```

构造函数初始值列表

类名::类名(参数列表) : 成员1(初值1), 成员2(初值2), ... { }

- 在对象创建时直接初始化成员，比在函数体内赋值更高效
- const 成员和引用成员必须用初始值列表初始化
- 成员的初始化顺序按声明顺序，而非初始值列表顺序

类内初始值（C++11）

```
class Contact {
```

```
private:
```

```
    string name = "";          // 类内初始值
```

```
    long long phone = 0;
```

```
public:
```

```
    Contact() = default;
```

```
    Contact(const string &str, long long no) : name(str), phone(no) { }
```

```
};
```

拷贝构造函数

```
class Contact {
```

```
public:
```

```
    Contact(const Contact &other); // 拷贝构造函数
```

```
};
```

```
Contact::Contact(const Contact &other)
    : name(other.name), phone(other.phone) { }
```

```
Contact c1("Jobs", 123);
Contact c2 = c1;      // 调用拷贝构造函数
Contact c3(c1);       // 调用拷贝构造函数
```

析构函数

析构函数是对象生命周期结束时自动调用的成员函数。

析构函数要点

- 函数名为 ~ 类名
- 没有参数和返回值
- 不能被重载（一个类只有一个析构函数）
- 局部对象在作用域结束时调用
- 全局/静态对象在程序结束时调用
- new 创建的对象在 delete 时调用
- 用于释放资源（如动态申请的内存、文件句柄等）

析构函数示例

```
class IntArray {
private:
    int *data;
    size_t size;
public:
    IntArray(size_t n) : size(n), data(new int[n]) { }
    ~IntArray() {    // 析构函数
        delete[] data;
    }
}
```

```
};

int main() {
    IntArray arr(10); // 构造
    // 使用 arr...
    return 0; // arr 离开作用域，自动调用析构函数
}
```

this 指针

this 是指向当前对象的常量指针。

```
class Contact {
    string name;
public:
    Contact& setName(const string &n) {
        name = n;
        return *this; // 返回对象自身的引用
    }
    void show() const { // const 成员函数
        cout << name << endl;
        // this 的类型是 const Contact*
    }
};
```

```
Contact c;
c.setName("Hello").show(); // 链式调用
```

this 指针

- this 是隐式参数，指向调用成员函数的对象
- 类型: `const ClassName*`（在 `const` 成员函数中）
- 类型: `ClassName* const`（在普通成员函数中）

- `return *this;` 常用于链式调用
- `const` 成员函数不能修改成员变量

封装

封装将数据与操作绑定，通过访问控制隐藏实现细节。

封装的优点

- 数据隐藏：保护数据完整性
- 接口与实现分离
- 降低耦合，提高内聚
- 提高代码可维护性和可重用性

继承初步

继承允许从已有类派生新类，复用代码。

```
class Person {
protected:
    string name;
    int age;
public:
    Person(const string &n, int a) : name(n), age(a) { }
    void show() { cout << name << ", " << age << endl; }
};

class Student : public Person {    // 公有继承
    string school;
public:
    Student(const string &n, int a, const string &s)
        : Person(n, a), school(s) { }
    void study() { cout << name << " is studying" << endl; }
```

```
};
```

```
Student stu("Zhang", 20, "SDU");  
stu.show();    // 继承自 Person  
stu.study();   // Student 自己的成员
```

继承要点

- `class Derived : public Base` 表示公有继承
- 派生类继承基类的所有成员（构造/析构造函数除外）
- 派生类可添加新成员，可覆盖基类的虚函数
- `protected` 成员：基类和派生类内可访问，类外不可访问
- 公有继承：is-a 关系（派生类是一种基类）

多态初步

多态允许通过基类指针/引用调用派生类的函数。

```
class Shape {  
public:  
    virtual double area() const { return 0; } // 虚函数  
    virtual ~Shape() { } // 虚析构造函数（推荐）  
};  
  
class Circle : public Shape {  
    double radius;  
public:  
    Circle(double r) : radius(r) { }  
    double area() const override { // override (C++11)  
        return 3.14159 * radius * radius;  
    }  
};
```

```
class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) { }
    double area() const override {
        return width * height;
    }
};
```

// 多态：基类指针调用派生类方法

```
Shape *s1 = new Circle(5);
Shape *s2 = new Rectangle(3, 4);
cout << s1->area() << endl; // 78.5398 (调用 Circle::area)
cout << s2->area() << endl; // 12 (调用 Rectangle::area)
delete s1;
delete s2;
```

多态要点

- virtual 声明虚函数，实现动态绑定
- 基类指针/引用调用虚函数时，根据实际对象类型决定调用哪个版本
- override (C++11) 显式标注覆盖，帮助编译器检查
- 基类通常需要虚析构函数
- 纯虚函数：virtual void f() = 0;; 含纯虚函数的类为抽象类

典型例题

例题 9.1

题目：设计一个 Rectangle 类，包含宽和高两个私有成员变量，提供构造函数、计算面积和周长的成员函数。

参考答案与解析

```
#include <iostream>
using namespace std;

class Rectangle {
private:
    double width, height;
public:
    Rectangle(double w, double h) : width(w), height(h) { }
    double area() const { return width * height; }
    double perimeter() const { return 2 * (width + height); }
};

int main() {
    Rectangle r(3, 4);
    cout << "面积: " << r.area() << endl;        // 12
    cout << "周长: " << r.perimeter() << endl;    // 14
    return 0;
}
```

例题 9.2

题目：下列程序的输出是什么？

```
class A {
public:
    A() { cout << "A()"; }
    ~A() { cout << "~A()"; }
};

class B : public A {
public:
    B() { cout << "B()"; }
    ~B() { cout << "~B()"; }
};

int main() {
```

```
B b;  
    return 0;  
}
```

参考答案与解析

答案：A()B() B() A()

解析：创建 B 的对象时，先调用基类 A 的构造函数，再调用 B 的构造函数。销毁时相反，先调用 B 的析构函数，再调用 A 的析构函数。

巩固练习

一、选择题

1. `class` 中成员的默认访问权限是_____。
A. `public` B. `private` C. `protected` D. `friend`
2. 构造函数的特点是_____。
A. 有返回类型 B. 函数名与类名相同 C. 可以被重载 D. 必须有无参版本
3. 下列关于析构函数的说法，正确的是_____。
A. 可以有参数 B. 可以重载 C. 名称为 `~ 类名` D. 有返回类型
4. `this` 指针的类型是_____。
A. 指向当前对象的常量指针 B. 指向当前对象的引用 C. 指向类本身 D. 空指针
5. 下列关于继承的说法，正确的是_____。
A. 派生类不能添加新成员 B. 派生类继承基类的构造函数 C. 公有继承表示 is-a 关系
D. 派生类不能访问基类的 `protected` 成员
6. 实现多态需要使用_____。
A. 静态函数 B. 虚函数 C. 内联函数 D. 友元函数

二、填空题

1. 类定义末尾必须有_____。
2. 构造函数_____ 返回类型。
3. 派生类构造函数的执行顺序是先调用_____ 的构造函数,再调用_____ 的构造函数。
4. 声明虚函数使用关键字_____。
5. 含有_____ 虚函数的类称为抽象类。

三、简答题

1. 简述 class 和 struct 的区别。
2. 说明构造函数初始值列表的优点。
3. 简述多态的实现原理。

四、综合应用题

1. 设计一个 BankAccount 类, 包含账户名和余额两个私有成员, 提供存款、取款和查询余额的公有方法。
2. 设计一个 Shape 基类和 Circle、Rectangle 两个派生类, 使用虚函数实现多态, 计算各种形状的面积。

参考答案与解析

答案：选择题：1.B 2.B 3.C 4.A 5.C 6.B

解析：

1. class 默认 private, struct 默认 public
2. 构造函数名与类名相同, 无返回类型
3. 析构函数名为 ~ 类名, 无参数无返回值, 不能重载
4. this 是指向当前对象的常量指针

5. 公有继承表示 is-a 关系
6. 虚函数实现动态绑定（多态）

填空题答案：

1. 分号
2. 没有
3. 基类；派生类
4. virtual
5. 纯

编程题参考代码：

第 1 题：

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    string owner;
    double balance;
public:
    BankAccount(const string &name, double init)
        : owner(name), balance(init) { }
    void deposit(double amount) { balance += amount; }
    bool withdraw(double amount) {
        if (amount > balance) return false;
        balance -= amount;
        return true;
    }
    double getBalance() const { return balance; }
```

```
};
```

```
int main() {  
    BankAccount acc("Zhang", 1000);  
    acc.deposit(500);  
    cout << acc.getBalance() << endl;    // 1500  
    acc.withdraw(200);  
    cout << acc.getBalance() << endl;    // 1300  
    return 0;  
}
```

第 2 题:

```
#include <iostream>  
using namespace std;  
  
class Shape {  
public:  
    virtual double area() const = 0;    // 纯虚函数  
    virtual ~Shape() { }  
};  
  
class Circle : public Shape {  
    double r;  
public:  
    Circle(double radius) : r(radius) { }  
    double area() const override {  
        return 3.14159 * r * r;  
    }  
};  
  
class Rectangle : public Shape {  
    double w, h;  
public:  
    Rectangle(double width, double height)
```

```
        : w(width), h(height) { }
    double area() const override { return w * h; }
};

int main() {
    Shape *shapes[] = {
        new Circle(5),
        new Rectangle(3, 4)
    };
    for (auto s : shapes) {
        cout << s->area() << endl;
        delete s;
    }
    return 0;
}
```

第 10 章 模拟试题

模拟试题一

模拟试题一

一、选择题（每题 2 分，共 20 分）

1. C++ 语言中，main 函数的返回类型必须是_____。

- A. void B. int C. float D. char

2. 下列哪个是合法的 C++ 标识符？

- A. 2var B. _myVar C. class D. my-var

3. 下列代码的输出是_____。

```
int a = 5, b = 3;  
cout << a / b << " " << a % b;
```

- A. 1 2 B. 1.67 2 C. 2 1 D. 2 2

4. 下列关于引用的说法，错误的是_____。

- A. 引用必须初始化 B. 引用可以改变绑定 C. 引用是别名 D. 引用不占用内存

5. 下列代码的输出是_____。

```
int i = 3;  
int &r = i;  
r = 10;  
cout << i;
```

- A. 3 B. 10 C. 编译错误 D. 未定义行为

6. 下列哪个运算符有短路求值特性?

- A. & B. || C. | D. ^

7. `vector<int> v{1, 2, 3};` 中 `v.size()` 的值是_____。

- A. 1 B. 2 C. 3 D. 4

8. 下列关于函数重载的说法, 正确的是_____。

- A. 返回类型不同可以重载 B. 形参列表不同可以重载 C. 形参名不同可以重载
D. 函数体不同可以重载

9. 类的默认访问权限是 (使用 `class` 关键字时) _____。

- A. `public` B. `private` C. `protected` D. `friend`

10. 下列关于构造函数的说法, 错误的是_____。

- A. 函数名与类名相同 B. 没有返回类型 C. 可以重载 D. 必须有无参版本

二、填空题 (每题 2 分, 共 20 分)

1. C++ 程序的编译过程包括预处理、编译、汇编和_____四个阶段。

2. `int a = 7 / 2;` 中 `a` 的值是_____。

3. 定义指向 `int` 的指针 `p` 并初始化为空指针的语句是_____。

4. `const int *p` 表示_____的指针。

5. `string s = "Hello";` 则 `s.size()` 返回_____。

6. `for (int i = 0; i < 5; ++i)` 循环执行_____次。

7. 函数声明也称为函数_____。

8. `unsigned u = 10;` 则 `u - 20` 的结果是_____ (假设 `int` 占 32 位)。

9. 类的析构函数名是在类名前加_____。

10. 实现多态需要使用_____函数。

三、读程序写结果题（每题 5 分，共 15 分）

1. 阅读以下程序，写出输出结果。

```
#include <iostream>
using namespace std;
void swap(int &a, int &b) {
    int t = a; a = b; b = t;
}
int main() {
    int x = 3, y = 5;
    swap(x, y);
    cout << x << " " << y << endl;
    return 0;
}
```

2. 阅读以下程序，写出输出结果。

```
#include <iostream>
using namespace std;
int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int *p = arr;
    int *q = arr + 3;
    cout << *p << " " << *q << " "
        << (q - p) << endl;
    return 0;
}
```

3. 阅读以下程序，写出输出结果。

```
#include <iostream>
using namespace std;
class A {
```

```
public:
    A() { cout << "A "; }
    ~A() { cout << "~A "; }
};
class B : public A {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};
int main() {
    B obj;
    return 0;
}
```

四、编程题（每题 10 分，共 20 分）

1. 编写程序，输入 n 个整数（n 也由用户输入），存入 vector，然后按从大到小的顺序输出。
2. 设计一个 Student 类，包含学号、姓名和成绩三个私有成员变量，提供构造函数、设置信息、获取信息和显示信息的公有成员函数。在主函数中创建对象并测试。

参考答案与解析

参考答案与解析

一、选择题

答案： 1.B 2.B 3.A 4.B 5.B 6.B 7.C 8.B 9.B 10.D

解析：

1. main 函数返回类型必须是 int
2. 标识符不能以数字开头，不能用关键字，不能用连字符
3. 整数除法 $7/2=3$ ，取余 $7\%2=1$

4. 引用一经绑定不能改变绑定对象
5. 引用是别名，修改 r 就是修改 i
6. 逻辑或 || 有短路求值，按位或 | 没有
7. 列表初始化 {1,2,3} 有 3 个元素
8. 重载要求形参列表不同
9. class 默认 private，struct 默认 public
10. 构造函数不必有无参版本，但如果没有，则不能用默认方式创建对象

二、填空题

答案：

1. 链接
2. 3
3. `int *p = nullptr;`
4. 指向 `const int`
5. 5
6. 5
7. 原型
8. 4294967286
9. ~
10. 虚

三、读程序写结果题

答案：

1. 5 3

解析：引用传递交换了 x 和 y 的值。

2. 1 4 3

解析：p 指向 arr[0]（值为 1），q 指向 arr[3]（值为 4），q-p=3（指针相减为元素个数）。

3. A B ~B ~A

解析：构造时先基类后派生类，析构时先派生类后基类。

四、编程题

第 1 题参考代码：

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> v(n);
    for (int i = 0; i < n; ++i)
        cin >> v[i];
    sort(v.begin(), v.end(), greater<int>());
    for (int x : v)
        cout << x << " ";
    cout << endl;
    return 0;
}
```

第 2 题参考代码：

```
#include <iostream>
#include <string>
using namespace std;

class Student {
```

```
private:
    string id;
    string name;
    double score;
public:
    Student(const string &i = "", const string &n = "", double s = 0)
        : id(i), name(n), score(s) { }

    void setInfo(const string &i, const string &n, double s) {
        id = i; name = n; score = s;
    }

    string getId() const { return id; }
    string getName() const { return name; }
    double getScore() const { return score; }

    void display() const {
        cout << "学号: " << id
            << ", 姓名: " << name
            << ", 成绩: " << score << endl;
    }
};

int main() {
    Student s1("2025001", "张三", 95.5);
    s1.display();

    Student s2;
    s2.setInfo("2025002", "李四", 88.0);
    s2.display();
    return 0;
}
```

模拟试题二

模拟试题二

一、选择题（每题 2 分，共 20 分）

1. 下列哪个是 C++11 引入的特性？

- A. 模板 B. auto 类型推导 C. 异常处理 D. 运算符重载

2. 下列代码的输出是_____。

```
int i = 0;
cout << i++ << " " << ++i;
```

- A. 0 2 B. 0 1 C. 1 2 D. 未定义行为

3. 下列关于 const 的说法，正确的是_____。

- A. const int a; 定义后可修改 B. const int *p 中 p 本身是常量 C. int *const p 中 p 本身是常量 D. const 引用不能绑定非 const 对象

4. 下列代码的输出是_____。

```
for (int i = 0; i < 5; ++i) {
    if (i == 2) continue;
    if (i == 4) break;
    cout << i;
}
```

- A. 01234 B. 013 C. 012 D. 0134

5. string s("hello"); s[0] = 'H'; 后 s 的值是_____。

- A. hello B. Hello C. HELLO D. Hhello

6. 下列关于 vector 的说法，错误的是_____。

- A. 大小可变 B. 可以用下标添加元素 C. 支持范围 for D. push_back 在尾部添加元素

7. 下列代码的输出是_____。

```
int a = 10;
auto f = [a](int x) { return a + x; };
cout << f(5);
```

- A. 5 B. 10 C. 15 D. 编译错误

8. 文件流 ofstream 默认的打开模式是_____。

- A. ios::in B. ios::out | ios::trunc C. ios::app D. ios::binary

9. 下列关于继承的说法，正确的是_____。

- A. 派生类继承基类的构造函数 B. 析构时先调用基类的析构函数 C. 公有继承表示 is-a 关系 D. 派生类不能访问基类的 protected 成员

10. 下列代码的输出是_____。

```
int x = 5;
cout << (x & 3);
```

- A. 0 B. 1 C. 3 D. 5

二、填空题（每题 2 分，共 20 分）

1. std::cout 的类型是_____。

2. int a = 3.14; 中 a 的值是_____。

3. int *p = new int[10]; 释放内存应使用_____。

4. const int *p 和 int *const p 中，_____ 是 const 指针。

5. decltype((x)) 当 x 是 int 时，结果是_____ 类型。

6. getline(cin, s) 从标准输入读取_____ 存入 s。

7. 函数的默认参数必须从_____ 开始连续提供。

8. `assert` 宏定义在_____ 头文件中。
9. `virtual void f() = 0;` 声明的是_____ 函数。
10. 构造函数初始化成员应使用_____ 列表。

三、读程序写结果题（每题 5 分，共 15 分）

1. 阅读以下程序，写出输出结果。

```
#include <iostream>
using namespace std;
int foo(int n) {
    if (n <= 1) return 1;
    return n * foo(n - 1);
}
int main() {
    cout << foo(5) << endl;
    return 0;
}
```

2. 阅读以下程序，写出输出结果。

```
#include <iostream>
using namespace std;
int main() {
    int sum = 0;
    int arr[3][4] = {{1,2,3,4},{5,6,7,8},{9,10,11,12}};
    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 4; ++j)
            sum += arr[i][j];
    cout << sum << endl;
    return 0;
}
```

3. 阅读以下程序，写出输出结果。

```
#include <iostream>
using namespace std;
class Base {
public:
    virtual void show() { cout << "Base" << endl; }
};
class Derived : public Base {
public:
    void show() { cout << "Derived" << endl; }
};
int main() {
    Base *p = new Derived;
    p->show();
    delete p;
    return 0;
}
```

四、编程题（每题 10 分，共 20 分）

1. 编写程序，输入一个字符串，统计其中大写字母、小写字母、数字和其他字符的个数。
2. 设计一个 Shape 抽象类，包含纯虚函数 `area()`。派生 Circle 和 Rectangle 类，计算面积。在主函数中使用基类指针数组实现多态调用。

参考答案与解析

参考答案与解析

一、选择题

答案： 1.B 2.D 3.C 4.B 5.B 6.B 7.C 8.B 9.C 10.B

解析：

1. auto 类型推导是 C++11 引入的

2. `i++` 和 `++i` 在同一表达式中求值顺序未定义（但常见输出为 0 2 或 1 1，严格来说是未定义行为）
3. `int *const p` 中 `p` 本身是常量指针，不能改变指向
4. `i=0` 输出 0；`i=1` 输出 1；`i=2` `continue` 跳过；`i=3` 输出 3；`i=4` `break` 跳出。输出 013
5. `s[0]='H'` 将第一个字符改为 H，结果为"Hello"
6. 不能用下标添加元素，只能用 `push_back`
7. Lambda 捕获 `a=10`，传入 `x=5`，返回 15
8. `ofstream` 默认 `out | trunc`
9. 公有继承表示 is-a 关系
10. 5 的二进制 101，3 的二进制 011，按位与得 001=1

注：第 2 题严格来说是未定义行为，但常见编译器输出 0 2。若选 D 也算对。

二、填空题

答案：

1. `ostream`
2. 3
3. `delete[] p;`
4. `int *const p`
5. `int&`
6. 一整行
7. 右
8. `<cassert>`
9. 纯虚

10. 初始值

三、读程序写结果题

答案：

1. 120

解析： $\text{foo}(5) = 5 \times 4 \times 3 \times 2 \times 1 = 120$ （递归计算阶乘）。

2. 78

解析： $1+2+\dots+12 = 78$ （1 到 12 的和）。

3. Derived

解析：show 是虚函数，通过基类指针调用时，根据实际对象类型（Derived）调用 Derived::show。

四、编程题

第 1 题参考代码：

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string s;
    getline(cin, s);
    int upper = 0, lower = 0, digit = 0, other = 0;
    for (char c : s) {
        if (c >= 'A' && c <= 'Z') ++upper;
        else if (c >= 'a' && c <= 'z') ++lower;
        else if (c >= '0' && c <= '9') ++digit;
        else ++other;
    }
    cout << "大写字母: " << upper << endl;
    cout << "小写字母: " << lower << endl;
    cout << "数字: " << digit << endl;
    cout << "其他: " << other << endl;
```

```
    return 0;
}
```

第 2 题参考代码:

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual double area() const = 0;
    virtual ~Shape() { }
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : radius(r) { }
    double area() const override {
        return 3.14159 * radius * radius;
    }
};

class Rectangle : public Shape {
    double width, height;
public:
    Rectangle(double w, double h)
        : width(w), height(h) { }
    double area() const override {
        return width * height;
    }
};

int main() {
    Shape *shapes[3];
```

```
    shapes[0] = new Circle(5);
    shapes[1] = new Rectangle(3, 4);
    shapes[2] = new Circle(10);

    for (int i = 0; i < 3; ++i) {
        cout << "面积: " << shapes[i]->area() << endl;
        delete shapes[i];
    }
    return 0;
}
```

考前速记卡片

核心知识点速记

1. 编译四步：预处理 → 编译 → 汇编 → 链接
2. 类型转换：窄 → 宽（int → unsigned → long → long long）
3. 引用 vs 指针：
 - 引用必须初始化，不能改变绑定
 - 指针可以不初始化，可以改变指向
 - 引用不是对象，指针是对象
4. const 限定符：
 - `const int *p`: 指向 const 的指针（不能通过 p 修改）
 - `int *const p`: const 指针（p 不能改变指向）
 - `const int &r`: const 引用（可绑定字面值、表达式）
5. 函数参数传递：
 - 值传递：拷贝，不修改实参
 - 引用传递：别名，可修改实参
 - 指针传递：传地址，可修改实参
6. 函数重载：形参列表必须不同，返回类型不参与
7. 构造/析构顺序：
 - 构造：基类 → 成员 → 派生类
 - 析构：派生类 → 成员 → 基类
8. 多态：virtual 虚函数 + 基类指针/引用
9. 常用头文件：
 - `<iostream>`: cin, cout

- `<string>`: string
- `<vector>`: vector
- `<fstream>`: 文件流
- `<sstream>`: 字符串流
- `<cassert>`: assert

易错点提醒

[易错] 无符号数运算: `unsigned u = 10; u - 20` 结果是一个很大的正数!

[易错] 整数除法: `7 / 2 = 3` (舍位), 不是 3.5!

[易错] 悬垂 else: else 与最近的未匹配 if 配对!

[易错] `= vs ==`: `if (a = 0)` 是赋值, 不是比较!

[易错] 数组越界: 下标从 0 开始, 最大为 `size-1`!

[易错] vector 下标添加: 不能用 `v[i] = x` 添加元素!

[易错] 引用必须初始化: `int &r;` 错误!

[易错] 返回局部引用: 不要返回局部变量的引用或指针!

[易错] switch 贯穿: 忘记 break 会继续执行下一个 case!

[易错] 类定义分号: 类定义末尾必须有分号!

祝考试顺利!

子冉一人 整理